.NET Core

ASP.NET Core: Basics to Advanced

Web APIs, DI, Middleware, EF Core, Security & Deployment (.NET 8/9/10)

- Hosting, Middleware pipeline, Dependency Injection, Configuration
- Controllers, Minimal APIs, Validation, Filters, Versioning
- EF Core, Auth (JWT, Identity, Policies), Caching, Background jobs
- Resilience, Testing, Performance, Docker/IIS & interview Q&A

Prepared by Kaif Shaikh • Study & Interview Edition

Table of Contents

1. Introduction to .NET Core and ASP.NET Core 5

1.1 How to Use This Guide 5
1.2 What is .NET Core? 5
1.3 .NET Architecture 6
1.4 The dotnet CLI 7
1.5 Project Structure 7
1.6 Program.cs: The Minimal Hosting Model (.NET 6+) 8
1.7 Hosting: Kestrel, IIS and Reverse Proxies 8
1.8 Generic Host and the App Lifecycle 9

2. Middleware and the Request Pipeline 10

2.1 What is Middleware? 10
2.2 Use, Run and Map 11
2.3 Writing a Custom Middleware Class 11
2.4 Recommended Middleware Order 12
2.5 Built-in Middleware Overview 13

3. Dependency Injection 14

3.1 Why Dependency Injection? 14
3.2 Service Lifetimes 15
3.3 Resolving Services 16
3.4 Multiple Implementations and Keyed Services 16
3.5 Disposal and Open Generics 16
3.6 Third-Party Containers and Scrutor 17

4. Configuration, Options and Logging 18

4.1 Configuration Sources 18
4.2 The Options Pattern 19
4.3 Logging with ILogger 20

5. Building Web APIs with Controllers 21

5.1 REST Basics 21
5.2 A Complete Controller 21
5.3 Routing 22
5.4 Model Binding 23
5.5 Validation 24
5.6 Action Results and Return Types 24
5.7 DTOs and Mapping 25
5.8 API Versioning and OpenAPI 25
5.9 JSON Serialization Settings 26

6. Minimal APIs 27

6.1 What are Minimal APIs? 27

6.2 Organising Endpoints: Route Groups and TypedResults 27

6.3 Endpoint Filters and Validation 28

6.4 Controllers vs Minimal APIs 28

7. Filters and Error Handling 29

7.1 MVC Filter Pipeline 29

7.2 Global Error Handling with ProblemDetails 30

7.3 Exceptions vs Result Objects 31

8. Entity Framework Core 32

8.1 What is EF Core? 32

8.2 Migrations (Code First) 34

8.3 Querying 34

8.4 Loading Related Data 35

8.5 Saving Data and Change Tracking 35

8.6 Transactions and Concurrency 36

8.7 DbContext Rules and Performance 36

9. Authentication, Authorization and Security 38

9.1 Authentication vs Authorization 38

9.2 JWT Bearer Authentication 38

9.3 Authorization: Roles, Claims and Policies 39

9.4 ASP.NET Core Identity 40

9.5 CORS 40

9.6 Rate Limiting (.NET 7+) 41

9.7 Security Checklist 41

10. Caching and Performance 42

10.1 Caching Options 42

10.2 Async All the Way 43

10.3 Performance Checklist 43

11. Background Services, HttpClient and Resilience 44

11.1 Hosted Services and BackgroundService 44

11.2 IHttpClientFactory 45

11.3 Resilience with Polly (Microsoft.Extensions.Http.Resilience) 46

12. Real-Time, gRPC, Health Checks and More 48

12.1 SignalR (Real-Time Communication) 48

12.2 gRPC 48

12.3 Health Checks 49

12.4 Other Useful Features 50

13. Testing 51

13.1 Testing Pyramid 51
13.2 Unit Testing with xUnit and Moq 51
13.3 Integration Testing with WebApplicationFactory 52
14. Architecture and Design Patterns 53
14.1 Clean Architecture 53
14.2 Repository and Unit of Work 54
14.3 CQRS and MediatR 54
14.4 Other Patterns You'll Meet 56
14.5 Microservices Basics 56
15. Deployment 57
15.1 Publish Options 57
15.2 Hosting on IIS (Windows) 57
15.3 Docker 58
15.4 Cloud and CI/CD 58
15.5 Production Readiness Checklist 59
16. Interview Questions and Tricky Scenarios 60
16.1 Scenario-Based Puzzles 60
16.2 Frequently Asked Questions 63
17. Quick Revision Cheat Sheet 65
17.1 Program.cs Template 65
17.2 Top 25 Tricky Points 65
17.3 Study Roadmap 66

1. Introduction to .NET Core and ASP.NET Core

1.1 How to Use This Guide

> **✎ Definition**
>
> A study guide that explains each ASP.NET Core topic in the same order: a simple explanation, a working code example, then the tricky points and interview tips that go with it.

This guide covers modern .NET (".NET Core") and ASP.NET Core from the first project to production topics: hosting, middleware, dependency injection, Web APIs, EF Core, security, caching, background services, testing and deployment. Every topic follows **explanation, then diagram/code, then tricky points**.

> **☼ Simple Analogy**
>
> A real-world comparison that makes the idea stick.

> **⚡ Tricky Point**
>
> Tricky points: behaviour that surprises people and is loved by interviewers.

> **★ Interview Tip**
>
> Interview tip: how to answer the question in one or two crisp lines.

> **⚠ Watch Out**
>
> Watch out: common production bugs and bad practices.

1.2 What is .NET Core?

> **✎ Definition**
>
> **.NET Core** (now just **.NET**, version 5 and later) is Microsoft's free, open-source, cross-platform runtime and framework. **ASP.NET Core** is the part of it for building web apps and APIs.

.NET Core is Microsoft's **free, open-source, cross-platform** runtime and framework for building web, cloud, desktop, mobile, IoT and console apps. Since .NET 5 the "Core" name was dropped from the runtime (it is now just ".NET 8", ".NET 9", ".NET 10"), but people still say ".NET Core" to distinguish it from the old Windows-only **.NET Framework**.

ASP.NET Core is the web framework on top of .NET for building Web APIs, MVC apps, Razor Pages, Blazor, SignalR real-time apps, gRPC services and minimal APIs.

Feature	.NET Framework 4.x	.NET Core / .NET 5+
Platforms	Windows only	Windows, Linux, macOS, containers
Open source	Reference source only	Fully open source (GitHub)
Web framework	ASP.NET (System.Web, IIS-bound)	ASP.NET Core (Kestrel, any host)
Performance	Good	Among the fastest web frameworks (TechEmpower)
Deployment	Machine-wide install	Side-by-side versions, self-contained, single file, Native AOT
DI, config, logging	Third-party/add-ons	Built in
Future	Maintenance/security fixes only	All new features

Release cadence and support

Version	Released	Type	Support ends
.NET 6	Nov 2021	LTS	Nov 2024 (ended)
.NET 7	Nov 2022	STS	May 2024 (ended)
.NET 8	Nov 2023	LTS	Nov 2026
.NET 9	Nov 2024	STS (24 months)	Nov 2026
.NET 10	Nov 2025	LTS	Nov 2028

A new major version ships every November. **Even numbers are LTS** (3 years), odd numbers are STS (now 24 months). Production systems usually target the latest LTS.

> **★ Interview Tip**
>
> ".NET Core vs .NET Framework?" - .NET Core (now .NET 5+) is the modern, cross-platform, open-source, high-performance successor with built-in DI, configuration and logging, and side-by-side deployment. .NET Framework is Windows-only and receives only maintenance updates.

1.3 .NET Architecture

> **✎ Definition**
>
> The layers of .NET: the **runtime** (CLR: JIT, GC, type system), the **Base Class Library**, **app models** such as ASP.NET Core and EF Core, and the **SDK** tooling that builds and runs apps.

- **Runtime (CoreCLR):** JIT compiler, garbage collector, type system, threading.
- **Base Class Library:** collections, IO, networking, LINQ, JSON, crypto.
- **SDK:** compilers (Roslyn), the `dotnet` CLI, MSBuild, NuGet.
- **ASP.NET Core shared framework:** Kestrel, middleware, MVC, Razor, SignalR.
- **NuGet:** package manager for libraries (EF Core, Serilog, Polly, MediatR...).

C# compiles to **IL** in assemblies (.dll); at runtime the JIT turns IL into native code (Tiered compilation, Dynamic PGO). **Native AOT** compiles ahead of time for fast start-up and small memory (great for microservices and serverless).

1.4 The dotnet CLI

✎ Definition

The **dotnet CLI** is the cross-platform command-line tool for creating, building, running, testing, publishing and managing packages of .NET projects (`dotnet new`, `build`, `run`, `test`, `publish`).

```
dotnet --list-sdks                      // installed SDKs
dotnet new list                         // available templates
dotnet new webapi -n Recon.Api --use-controllers   // Web API with controllers
dotnet new webapi -n Recon.MinimalApi   // minimal API (default template)
dotnet new worker -n Recon.Worker       // background worker service
dotnet new classlib -n Recon.Domain
dotnet new xunit -n Recon.Tests
dotnet new sln -n Recon && dotnet sln add **/*.csproj
dotnet add Recon.Api reference Recon.Domain
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet restore / build / run / test
dotnet watch run                        // hot reload while developing
dotnet publish -c Release -o ./publish  // deployable output
```

1.5 Project Structure

✎ Definition

The files and folders of an ASP.NET Core project: the `.csproj` project file, `Program.cs` (entry point), `appsettings.json` (configuration), `Properties/launchSettings.json`, and folders such as Controllers and Models.

```
Recon.Api/
  Recon.Api.csproj          // SDK-style project: target framework, packages
  Program.cs                // entry point: builder, services, middleware pipeline
  appsettings.json          // configuration
  appsettings.Development.json
  Properties/launchSettings.json   // local profiles: URLs, environment variables
  Controllers/              // API controllers
  Models/ or Dtos/          // request/response contracts
  Services/                 // business logic
  Data/                     // DbContext, migrations
```

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.*" />
  </ItemGroup>
</Project>
```

⚡ Tricky Point

`launchSettings.json` is used **only** by Visual Studio/`dotnet run` on your machine. It is **not** deployed. Set `ASPNETCORE_ENVIRONMENT` and URLs on the server via environment variables, IIS config or Docker.

1.6 Program.cs: The Minimal Hosting Model (.NET 6+)

✎ Definition

The **minimal hosting model** configures and starts the whole app in `Program.cs`: `WebApplication.CreateBuilder` registers services, then `builder.Build()` creates the app where middleware and endpoints are added before `app.Run()`.

```
var builder = WebApplication.CreateBuilder(args);   // 1. host: config, logging, DI container, Kestrel

// 2. Register services (DI)
builder.Services.AddControllers();
builder.Services.AddOpenApi();                      // .NET 9+ (AddSwaggerGen in earlier templates)
builder.Services.AddDbContext<ReconDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Recon")));
builder.Services.AddScoped<IReconService, ReconService>();

var app = builder.Build();                          // 3. build the app
// 4. Configure the middleware pipeline (ORDER MATTERS)
if (app.Environment.IsDevelopment()) app.MapOpenApi();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();                               // 5. map endpoints

app.Run();                                          // 6. start Kestrel and listen
```

i Note

Before .NET 6, the same setup was split into `Program.cs` (`CreateHostBuilder`) and `Startup.cs` (`ConfigureServices` + `Configure`). You will still see that pattern in older projects and interviews; it works identically.

★ Interview Tip

"What does `WebApplication.CreateBuilder` do?" - It creates the host with default configuration sources (appsettings, environment variables, command line, user secrets), logging providers (console, debug), the DI container, and configures Kestrel and IIS integration.

1.7 Hosting: Kestrel, IIS and Reverse Proxies

✎ Definition

Kestrel is ASP.NET Core's built-in, cross-platform web server. In production it often runs behind a **reverse proxy** (IIS, Nginx, a load balancer) that forwards requests to it and handles TLS and routing.



IIS in-process: app runs inside w3wp.exe (fastest on Windows). Out-of-process / Linux: proxy forwards to Kestrel.

Figure 1.1 - Typical hosting setup

- **Kestrel** is ASP.NET Core's built-in, cross-platform, high-performance web server. Every ASP.NET Core app runs on Kestrel (or HTTP.sys on Windows).
- In production it often sits behind a **reverse proxy** (IIS, Nginx, Apache, Azure Front Door, YARP) for TLS termination, load balancing and static content.
- **IIS in-process hosting** (default): the app runs inside the IIS worker process via the ASP.NET Core Module - fastest option on Windows.
- **Out-of-process:** IIS forwards requests to a separate Kestrel process.

> ⚡ **Tricky Point**
>
> Behind a proxy, your app sees the proxy's IP and `http` scheme unless you add `app.UseForwardedHeaders()` (configured with `ForwardedHeadersOptions`). Symptoms: wrong redirect URLs, HTTPS redirect loops, client IP always 127.0.0.1.

1.8 Generic Host and the App Lifecycle

> ✎ **Definition**
>
> The **Generic Host** (`IHost`) owns the app's DI container, configuration, logging and hosted services, and manages its **lifecycle**: start, run, and graceful shutdown (`IHostApplicationLifetime`).

Both web apps and worker services run on the **Generic Host** (`IHost`), which provides DI, configuration, logging and lifetime management.

```
var app = builder.Build();
var lifetime = app.Lifetime;     // IHostApplicationLifetime
lifetime.ApplicationStarted.Register(() => Console.WriteLine("Started"));
lifetime.ApplicationStopping.Register(() => Console.WriteLine("Stopping - finish work"));
```

On shutdown (Ctrl+C, SIGTERM, IIS recycle), the host signals `ApplicationStopping`, stops accepting requests, waits for in-flight work up to `HostOptions.ShutdownTimeout` (30 s default in .NET 8+), and stops hosted services.

2. Middleware and the Request Pipeline

2.1 What is Middleware?

> ✎ **Definition**
>
> **Middleware** is a component in the request pipeline. Each one can handle an HTTP request, pass it to the next component, and act on the response on its way back.

Middleware is software assembled into a **pipeline** that handles every HTTP request and response. Each component can inspect/modify the request, call the **next** component, and inspect/modify the response on the way back - or **short-circuit** and respond immediately.

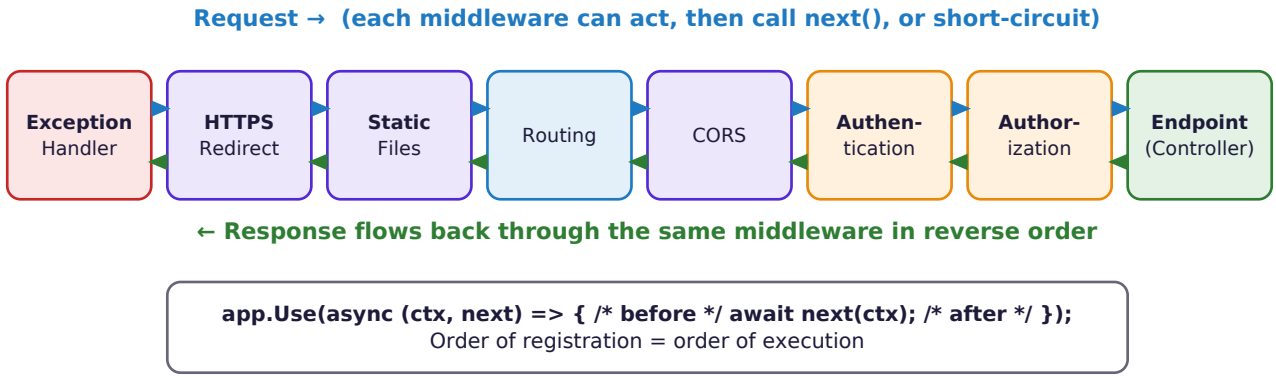


Figure 2.1 - The ASP.NET Core middleware pipeline

> ☼ **Simple Analogy**
>
> Airport security: you pass through a series of checkpoints (ID check, baggage scan, boarding gate). Any checkpoint can stop you. On arrival (the response) you pass through some checkpoints again in reverse.

2.2 Use, Run and Map

✎ Definition

Ways to add middleware: Use adds a component that can call the next one, Run adds a **terminal** component that ends the pipeline, and Map/MapWhen branch the pipeline by path or condition.

```
// Use: do something, then call the next middleware
app.Use(async (context, next) =>
{
    var sw = Stopwatch.StartNew();
    await next(context);                                    // call the rest of the pipeline
    app.Logger.LogInformation("{Path} took {Ms} ms", context.Request.Path, sw.ElapsedMilliseconds);
});

// Run: terminal middleware - never calls next
app.Run(async context => await context.Response.WriteAsync("End of pipeline"));
// Map: branch the pipeline by path
app.Map("/health-legacy", branch => branch.Run(ctx => ctx.Response.WriteAsync("OK")));
app.MapWhen(ctx => ctx.Request.Query.ContainsKey("debug"), b => b.UseMiddleware<DebugMiddleware>());
app.UseWhen(ctx => ctx.Request.Path.StartsWithSegments("/api"), b => b.UseMiddleware<ApiKeyMiddleware>());
```

⚡ Tricky Point

`UseWhen` rejoins the main pipeline after the branch; `MapWhen` does not (the branch must end with a terminal middleware). Anything registered **after** `app.Run(...)` never executes.

2.3 Writing a Custom Middleware Class

✎ Definition

A **middleware class** takes a `RequestDelegate next` in its constructor and has an `InvokeAsync(HttpContext)` method. It is registered with `app.UseMiddleware<T>()`.

```
public class CorrelationIdMiddleware
{
    private const string Header = "X-Correlation-Id";
    private readonly RequestDelegate _next;
    private readonly ILogger<CorrelationIdMiddleware> _logger;

    public CorrelationIdMiddleware(RequestDelegate next, ILogger<CorrelationIdMiddleware> logger)
    { _next = next; _logger = logger; }                     // ctor: singleton-like, created ONCE

    public async Task InvokeAsync(HttpContext context, IRequestContext requestCtx)  // scoped deps HERE
    {
        var id = context.Request.Headers[Header].FirstOrDefault() ?? Guid.NewGuid().ToString();
        requestCtx.CorrelationId = id;
        context.Response.Headers[Header] = id;
        using (_logger.BeginScope(new Dictionary<string, object> { ["CorrelationId"] = id }))
        {
            await _next(context);
        }
    }
}
// registration
app.UseMiddleware<CorrelationIdMiddleware>();
```

⚡ Tricky Point

Convention-based middleware is constructed **once** for the app lifetime. Injecting a **scoped** service (like `DbContext`) into its constructor creates a captive dependency. Inject scoped services as parameters of `InvokeAsync` instead, or implement `IMiddleware` (factory-based, resolved per request and registered in DI).

⚠ Watch Out

Don't modify headers or status code **after** the response has started (`context.Response.HasStarted`), e.g. after `await next()` when a downstream component already wrote the body. It throws `InvalidOperationException`. Use `context.Response.OnStarting(...)` to add headers safely.

2.4 Recommended Middleware Order

✎ Definition

The order in which middleware must be added, because the pipeline runs in registration order: exception handling, HTTPS, static files, routing, CORS, authentication, authorization, then endpoints.

```
app.UseExceptionHandler();       // 1. catch exceptions from everything after it
app.UseHsts();                   // 2. production only
app.UseHttpsRedirection();
app.UseStaticFiles();            // 3. short-circuits for files (no auth needed)
app.UseRouting();                // 4. select endpoint (implicit in .NET 6+ if omitted)
app.UseCors();                   // 5. after routing, before auth
app.UseAuthentication();         // 6. who are you?
app.UseAuthorization();          // 7. are you allowed?
app.UseRateLimiter();
app.UseResponseCaching();        // or UseOutputCache
app.MapControllers();            // 8. endpoints
```

★ Interview Tip

"Why does middleware order matter?" - Middleware runs in the order it's added. Authentication must run before authorization; the exception handler must be first to catch errors from later components; CORS must run before endpoints respond; static files placed early avoid unnecessary auth work.

2.5 Built-in Middleware Overview

✎ Definition

The ready-made middleware ASP.NET Core provides, such as exception handling, HTTPS redirection, static files, routing, CORS, authentication, authorization, response compression, rate limiting and output caching.

Middleware	Purpose
`UseExceptionHandler` / `UseDeveloperExceptionPage`	Global error handling / detailed errors in dev
`UseHttpsRedirection`, `UseHsts`	Force HTTPS
`UseStaticFiles`, `UseDefaultFiles`	Serve wwwroot files
`UseRouting`, `UseEndpoints` / `Map*`	Endpoint routing
`UseCors`	Cross-origin requests
`UseAuthentication`, `UseAuthorization`	Security
`UseRateLimiter`	Rate limiting (.NET 7+)
`UseResponseCompression`	Gzip/Brotli
`UseOutputCache`, `UseResponseCaching`	Caching responses
`UseRequestLocalization`	Culture per request
`UseForwardedHeaders`	Trust proxy headers
`UseHttpLogging` / `UseW3CLogging`	Request/response logging
`UseWebSockets`	WebSocket support

3. Dependency Injection

3.1 Why Dependency Injection?

> ✎ **Definition**
>
> **Dependency Injection** means a class receives the services it needs (usually through its constructor) instead of creating them. This gives loose coupling, easier testing and central configuration.

Dependency Injection (DI) means a class receives the objects it needs (its dependencies) from outside, usually through its constructor, instead of creating them with `new`. It implements the **Dependency Inversion Principle**: depend on abstractions (interfaces), not concrete classes.

```
// Without DI: tightly coupled, hard to test
public class ReconService
{
    private readonly SqlTxnRepository _repo = new SqlTxnRepository("Server=...");   // hard-coded
}

// With DI: loosely coupled, testable
public class ReconService(ITxnRepository repo, ILogger<ReconService> logger) : IReconService
{
    public async Task<int> RunAsync(DateOnly date)
    {
        var pending = await repo.GetUnmatchedAsync(date);
        logger.LogInformation("Found {Count} unmatched txns", pending.Count);
        return pending.Count;
    }
}
```

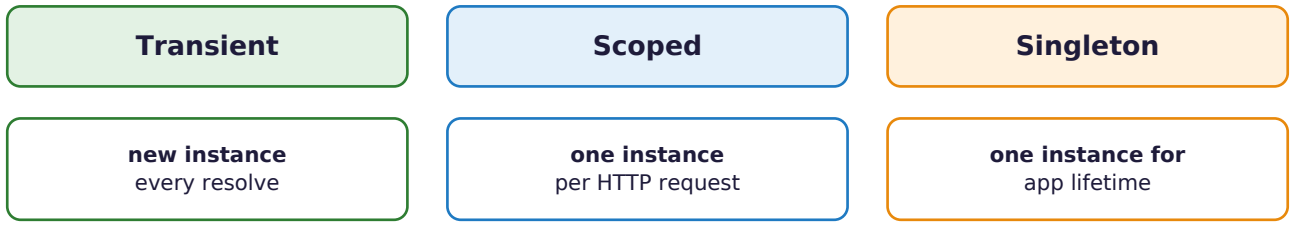
> ☼ **Simple Analogy**
>
> A restaurant chef doesn't grow vegetables or build an oven; the restaurant (container) provides them. If you change suppliers, the chef's recipe doesn't change. In tests, you hand the chef plastic vegetables (mocks).

Benefits: loose coupling, easy unit testing with mocks, centralised configuration of object lifetimes, swapping implementations (SQL vs in-memory, real vs fake gateway) without touching consumers.

3.2 Service Lifetimes

✎ Definition

How long a registered service instance lives: **Transient** (new every time it's requested), **Scoped** (one per HTTP request) and **Singleton** (one for the whole app).



Rule: a service may depend only on services with an EQUAL or LONGER lifetime.

Singleton → Scoped = "captive dependency" bug (scope validation throws in Development).

Figure 3.1 - Transient, Scoped and Singleton

```
builder.Services.AddTransient<IFeeCalculator, FeeCalculator>();      // new every time
builder.Services.AddScoped<ITxnRepository, SqlTxnRepository>();       // one per request
builder.Services.AddSingleton<IClock, SystemClock>();                  // one for the app

builder.Services.AddSingleton(new ReconSettings { BatchSize = 500 });  // existing instance
builder.Services.AddScoped<IReconService>(sp =>                         // factory
    new ReconService(sp.GetRequiredService<ITxnRepository>(),
        sp.GetRequiredService<ILogger<ReconService>>()));
builder.Services.TryAddScoped<ITxnRepository, SqlTxnRepository>();    // only if not registered yet
```

Lifetime	Use for	Watch out
Transient	Lightweight, stateless services	Many allocations; disposable transients are tracked until the scope ends
Scoped	DbContext, repositories, unit of work, per-request user context	Not available outside a scope (background services must create one)
Singleton	Caches, configuration, HttpClient handlers, expensive-to-create stateless services	Must be **thread-safe**; must not depend on scoped services

⚡ Tricky Point

Captive dependency: a singleton that takes a scoped service (e.g. `DbContext`) in its constructor keeps one instance forever - shared across all requests and threads. Result: stale data, `DbContext` concurrency exceptions, memory growth. ASP.NET Core's scope validation throws at start-up in Development to catch this.

★ Interview Tip

"Scoped vs Transient within one request?" - Scoped returns the **same** instance everywhere in that request; transient returns a **new** instance each time it's injected, even within the same request.

3.3 Resolving Services

✎ Definition

Ways to get services from the container: **constructor injection** (preferred), `[FromServices]` or Minimal API parameters, and `IServiceProvider/CreateScope()` when a scope is needed manually.

```
// 1) Constructor injection (preferred)
public class TxnController(IReconService recon) : ControllerBase { }

// 2) Action parameter injection
public IActionResult Get([FromServices] IClock clock) => Ok(clock.Now);

// 3) Minimal API parameters are injected automatically
app.MapGet("/now", (IClock clock) => clock.Now);

// 4) Manual scope (background services, startup tasks)
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<ReconDbContext>();
    await db.Database.MigrateAsync();
}
```

⚠ Watch Out

Avoid the **Service Locator** anti-pattern (`HttpContext.RequestServices.GetService<T>()` or injecting `IServiceProvider` everywhere). It hides dependencies and makes code harder to test. Use it only in infrastructure code (factories, middleware).

3.4 Multiple Implementations and Keyed Services

✎ Definition

Registering several implementations of one interface: inject `IEnumerable<T>` to get all of them, or use **keyed services** (.NET 8) with `[FromKeyedServices("key")]` to pick one by name.

```
builder.Services.AddScoped<INotifier, EmailNotifier>();
builder.Services.AddScoped<INotifier, SmsNotifier>();
public class AlertService(IEnumerable<INotifier> notifiers) { }   // gets BOTH
public class Other(INotifier notifier) { }                         // gets the LAST registered (Sms)

// .NET 8 keyed services
builder.Services.AddKeyedScoped<IPaymentGateway, VisaGateway>("visa");
builder.Services.AddKeyedScoped<IPaymentGateway, RupayGateway>("rupay");
public class Checkout([FromKeyedServices("visa")] IPaymentGateway gateway) { }
```

3.5 Disposal and Open Generics

✎ Definition

The container **disposes** the `IDisposable` services it created when their scope ends. **Open generics** register one generic type for every type argument, e.g. `IRepository<>` with `Repository<>`.

```
builder.Services.AddScoped(typeof(IRepository<>), typeof(EfRepository<>));   // open generic
```

The container disposes `IDisposable`/`IAsyncDisposable` services it created at the end of their lifetime (scope end for scoped/transient, app shutdown for singletons). It does **not** dispose instances you passed in yourself (`AddSingleton(instance)`).

3.6 Third-Party Containers and Scrutor

> ✎ **Definition**
>
> Replacement or add-on DI containers (Autofac and others) for advanced features, and **Scrutor**, a library that adds assembly scanning and decoration to the built-in container.

The built-in container covers most needs. Autofac/Lamar add property injection, modules and decorators. **Scrutor** adds assembly scanning and decoration on top of the built-in container:

```
builder.Services.Scan(s => s.FromAssemblyOf<IReconService>()
    .AddClasses(c => c.AssignableTo<IFeeRule>()).AsImplementedInterfaces().WithScopedLifetime());
builder.Services.Decorate<ITxnRepository, CachedTxnRepository>();     // decorator pattern
```

4. Configuration, Options and Logging

4.1 Configuration Sources

> ✎ **Definition**
>
> **Configuration** is a key-value system built from layered **sources**, where later sources override earlier ones: `appsettings.json`, `appsettings.{Environment}.json`, user secrets, environment variables, command-line args and Key Vault.

Configuration is a set of **key-value pairs** loaded from multiple providers, merged in order.

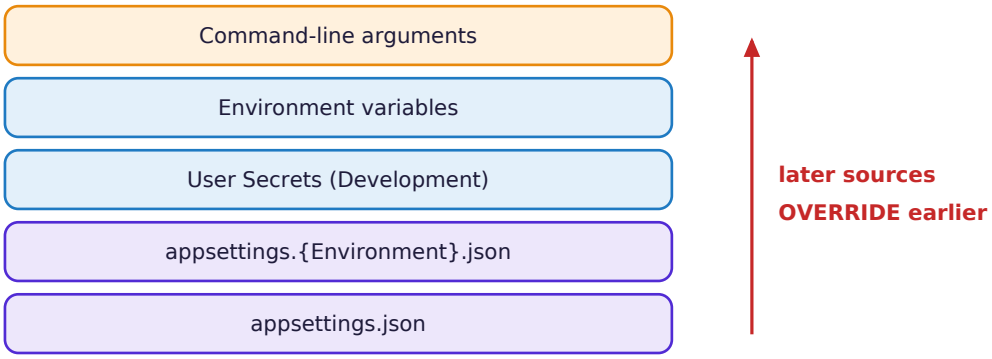


Figure 4.1 - Default configuration sources (later wins)

```
// appsettings.json
{
  "ConnectionStrings": { "Recon": "Server=.;Database=Recon;Trusted_Connection=True;TrustServerCertificate=True" },
  "Recon": { "BatchSize": 500, "Banks": [ "HDFC", "ICICI" ], "Sftp": { "Host": "sftp.bank.com",
      "Port": 22 } },
  "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }
}

// reading
string? cs = builder.Configuration.GetConnectionString("Recon");
int batch = builder.Configuration.GetValue<int>("Recon:BatchSize");          // ':' for nesting
string? host = builder.Configuration["Recon:Sftp:Host"];
```

Environment variables use **double underscore** for nesting: `Recon__Sftp__Host=prod.bank.com`. Arrays use indexes: `Recon__Banks__0=HDFC`.

> ⚡ **Tricky Point**
>
> Never commit secrets (passwords, keys) in `appsettings.json`. Use **User Secrets** in development (`dotnet user-secrets set "Jwt:Key" "..."`) and **environment variables / Azure Key Vault / AWS Secrets Manager** in production.

Environments

`ASPNETCORE_ENVIRONMENT` (or `DOTNET_ENVIRONMENT`) = `Development`, `Staging`, `Production` (default when unset). It selects `appsettings.{Environment}.json` and lets you branch: `app.Environment.IsDevelopment()`.

> **⚠ Watch Out**
>
> If `ASPNETCORE_ENVIRONMENT` is not set on the server, the app runs as **Production** - developer exception pages and Swagger (if guarded by IsDevelopment) won't appear. Conversely, accidentally deploying with `Development` leaks stack traces to users.

4.2 The Options Pattern

> **✎ Definition**
>
> The **Options pattern** binds a configuration section to a strongly typed class, which is injected as `IOptions<T>`, `IOptionsSnapshot<T>` or `IOptionsMonitor<T>` and can be validated at startup.

Bind a configuration section to a strongly typed class and inject it.

```
public class ReconOptions
{
    public const string Section = "Recon";
    [Range(1, 10_000)] public int BatchSize { get; set; }
    [Required] public string[] Banks { get; set; } = [];
    public SftpOptions Sftp { get; set; } = new();
}
public class SftpOptions { public string Host { get; set; } = ""; public int Port { get; set; } = 22; }

builder.Services.AddOptions<ReconOptions>()
    .Bind(builder.Configuration.GetSection(ReconOptions.Section))
    .ValidateDataAnnotations()
    .ValidateOnStart();                                  // fail fast at startup if config is invalid

public class ReconService(IOptions<ReconOptions> options)
{
    private readonly ReconOptions _opt = options.Value;
}
```

Interface	Lifetime	Reloads on change?	Use
`IOptions<T>`	Singleton	No (read once)	Most services
`IOptionsSnapshot<T>`	Scoped	Yes, per request	Per-request fresh values; named options
`IOptionsMonitor<T>`	Singleton	Yes, live + `OnChange` callback	Singletons/background services needing live config

> **⚡ Tricky Point**
>
> `IOptionsSnapshot<T>` is **scoped**, so it cannot be injected into a singleton (captive dependency). Use `IOptionsMonitor<T>.CurrentValue` there.

4.3 Logging with ILogger

✎ Definition

ILogger<T> is the built-in logging abstraction. It writes **structured** log messages at levels (Trace to Critical) to configured providers such as Console, Debug or Serilog.

```
public class SettlementService(ILogger<SettlementService> logger)
{
    public void Settle(string batchId, int count)
    {
        logger.LogInformation("Settling batch {BatchId} with {Count} txns", batchId, count);  // structured
        try { /* ... */ }
        catch (Exception ex)
        {
            logger.LogError(ex, "Settlement failed for {BatchId}", batchId);   // pass the exception first
        }
    }
}
```

Level	Use
Trace / Debug	Detailed diagnostics (disabled in production)
Information	Normal flow: request handled, job finished
Warning	Unexpected but handled: retry, missing optional data
Error	Operation failed
Critical	App-wide failure: DB down, out of disk

⚡ Tricky Point

Use **message templates**, not string interpolation: `LogInformation("User {Id}", id)` keeps `Id` as a structured, searchable property and avoids building the string when the level is disabled. `LogInformation($"User {id}")` loses both benefits (analyzer CA2254 warns).

High-performance logging (source generator)

```
public static partial class Log
{
    [LoggerMessage(Level = LogLevel.Information, Message = "Matched {Count} txns for {Bank}")]
    public static partial void Matched(ILogger logger, int count, string bank);
}
Log.Matched(logger, 1500, "HDFC");
```

Serilog (popular structured logging)

```
builder.Host.UseSerilog((ctx, cfg) => cfg
    .ReadFrom.Configuration(ctx.Configuration)
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.File("logs/recon-.log", rollingInterval: RollingInterval.Day));
app.UseSerilogRequestLogging();
```

i Note

For distributed systems add **OpenTelemetry** (traces, metrics, logs) exported to tools like Application Insights, Grafana, Jaeger or Seq. .NET Aspire wires this up automatically.

5. Building Web APIs with Controllers

5.1 REST Basics

> ✎ **Definition**
>
> **REST** is an architectural style for APIs: resources are identified by URLs, handled with standard HTTP verbs (GET, POST, PUT, PATCH, DELETE), are stateless, and return meaningful status codes.

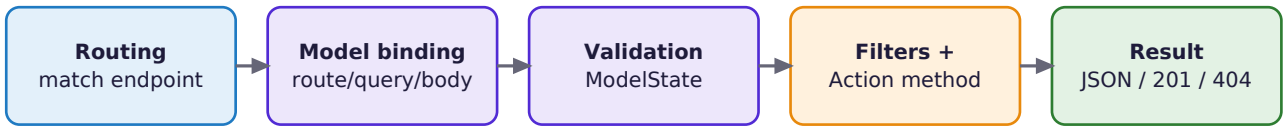
HTTP verb	Action	Success status	Idempotent?
GET /api/txns	List	200 OK	Yes
GET /api/txns/{id}	Read one	200 OK / 404 Not Found	Yes
POST /api/txns	Create	201 Created + Location header	No
PUT /api/txns/{id}	Replace	204 No Content / 200 OK	Yes
PATCH /api/txns/{id}	Partial update	204 / 200	Not necessarily
DELETE /api/txns/{id}	Delete	204 No Content	Yes

Common errors: **400** Bad Request (validation), **401** Unauthorized (not authenticated), **403** Forbidden (authenticated, not allowed), **404** Not Found, **409** Conflict, **422** Unprocessable Entity, **429** Too Many Requests, **500** Internal Server Error, **503** Service Unavailable.

> ⚡ **Tricky Point**
>
> 401 vs 403 is a classic question: **401 = "who are you?"** (missing/invalid token), **403 = "I know who you are, but you can't do this"** (insufficient role/policy).

5.2 A Complete Controller

> ✎ **Definition**
>
> A **controller** is a class (usually inheriting `ControllerBase` with `[ApiController]`) whose public **action methods** handle HTTP requests for one resource.



[ApiController] returns 400 ValidationProblemDetails automatically when ModelState is invalid.

Figure 5.1 - What happens when a request hits a controller

```
[ApiController]                                    // auto 400s, binding inference, ProblemDetails
[Route("api/[controller]")]                        // -> api/transactions
[Produces("application/json")]
public class TransactionsController(ITxnService service) : ControllerBase
{
    [HttpGet]                                      // GET api/transactions?page=1&status=FAILED
    public async Task<ActionResult<PagedResult<TxnDto>>> GetAll(
        [FromQuery] TxnQuery query, CancellationToken ct)
        => Ok(await service.SearchAsync(query, ct));
    [HttpGet("{id:long}", Name = nameof(GetById))]   // route constraint
    [ProducesResponseType<TxnDto>(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<TxnDto>> GetById(long id, CancellationToken ct)
    {
        var txn = await service.GetAsync(id, ct);
        return txn is null ? NotFound() : Ok(txn);
    }

    [HttpPost]
    public async Task<ActionResult<TxnDto>> Create(CreateTxnRequest request, CancellationToken ct)
    {
        var created = await service.CreateAsync(request, ct);
        return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);   // 201 + Location
    }

    [HttpPut("{id:long}")]
    public async Task<IActionResult> Update(long id, UpdateTxnRequest request, CancellationToken ct)
        => await service.UpdateAsync(id, request, ct) ? NoContent() : NotFound();
    [HttpDelete("{id:long}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Delete(long id, CancellationToken ct)
    {
        await service.DeleteAsync(id, ct);
        return NoContent();
    }
}
```

★ Interview Tip

`ControllerBase` is for APIs (no view support). `Controller` derives from it and adds views (`View()`, `PartialView()`), `TempData` and `ViewBag` for MVC apps.

5.3 Routing

✎ Definition

Routing maps an incoming URL and HTTP verb to an endpoint. Controllers use **attribute routing** (`[Route]`, `[HttpGet("{id:int}")]`) with route templates and constraints.

```
[Route("api/v1/banks/{bankId}/settlements")]     // attribute routing (APIs)
// constraints: int, long, guid, bool, datetime, min(1), regex(...)
[HttpGet("{date:datetime}")]
[HttpGet("search/{**path}")]                      // catch-all
[HttpGet("~/api/health")]                         // ~ ignores the controller route prefix
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");   // conventional (MVC)
```

⚡ Tricky Point

Route constraints are for **disambiguation**, not validation: a request to `/api/transactions/abc` with `{id:long}` returns **404**, not 400. Validate input in the model instead.

5.4 Model Binding

✎ Definition

Model binding automatically converts request data (route values, query string, headers, form and JSON body) into action parameters. Sources can be set with `[FromRoute]`, `[FromQuery]` and `[FromBody]`.

Source attribute	Reads from	Default inference with [ApiController]
`[FromRoute]`	Route values	Parameters matching route template names
`[FromQuery]`	Query string	Simple types not in the route
`[FromBody]`	JSON request body	Complex types (only one body parameter allowed)
`[FromForm]`	Form fields / files	`IFormFile`, `IFormFileCollection`
`[FromHeader]`	Request headers	Never inferred
`[FromServices]`	DI container	Registered services (inferred in .NET 7+)

```
public record TxnQuery(int Page = 1, int PageSize = 20, string? Status = null, DateOnly? From = null);

[HttpPost("upload")]
[RequestSizeLimit(50_000_000)]
public async Task<IActionResult> Upload(IFormFile file, [FromHeader(Name = "X-Bank")] string bank)
{
    if (file.Length == 0) return BadRequest("Empty file");
    await using var stream = file.OpenReadStream();
    var rows = await _parser.ParseAsync(stream, bank);
    return Ok(new { rows });
}
```

⚡ Tricky Point

With `[FromBody]`, the body stream can be read only once. A malformed JSON body or a type mismatch makes the parameter `null` / invalid and `[ApiController]` returns 400 automatically - check the `errors` object in the ProblemDetails response.

5.5 Validation

> ✎ **Definition**
>
> **Validation** checks that incoming models are correct, using Data Annotations (`[Required]`, `[Range]`) or FluentValidation. With `[ApiController]`, invalid models return 400 automatically.

```
public class CreateTxnRequest
{
    [Required, StringLength(12, MinimumLength = 12)] public string Rrn { get; set; } = "";
    [Range(0.01, 10_000_000)] public decimal Amount { get; set; }
    [Required, RegularExpression("^(VISA|MASTER|RUPAY)$")] public string Network { get; set; } = "";
    [EmailAddress] public string? NotifyEmail { get; set; }
}
// Custom rule across properties
public class DateRange : IValidatableObject
{
    public DateOnly From { get; set; } public DateOnly To { get; set; }
    public IEnumerable<ValidationResult> Validate(ValidationContext ctx)
    {
        if (To < From) yield return new ValidationResult("To must be after From", [nameof(To)]);
    }
}
```

Without `[ApiController]` you must check `if (!ModelState.IsValid) return ValidationProblem(ModelState);` yourself.

FluentValidation (popular alternative)

```
public class CreateTxnValidator : AbstractValidator<CreateTxnRequest>
{
    public CreateTxnValidator(ITxnRepository repo)
    {
        RuleFor(x => x.Rrn).NotEmpty().Length(12)
            .MustAsync(async (rrn, ct) => !await repo.ExistsAsync(rrn, ct)).WithMessage("Duplicate RRN");
        RuleFor(x => x.Amount).GreaterThan(0);
    }
}
```

5.6 Action Results and Return Types

> ✎ **Definition**
>
> What an action returns: a specific type, `IActionResult`, or `ActionResult<T>`, created with helpers such as `Ok()`, `NotFound()`, `CreatedAtAction()` and `NoContent()` to set the status code.

Return type	When
`IActionResult`	Multiple result types, no specific body type
`ActionResult<T>`	Return `T` or any `ActionResult` - best for Swagger/OpenAPI docs
Specific type `T`	Always 200 with T
`Results<Ok<T>, NotFound>` (TypedResults)	Minimal APIs - compile-time documented results
`IAsyncEnumerable<T>`	Stream large results (not buffered)

Helpers: `Ok()`, `Created()`, `CreatedAtAction()`, `NoContent()`, `BadRequest()`, `NotFound()`, `Conflict()`, `Unauthorized()`, `Forbid()`, `StatusCode(503)`, `File(bytes, "text/csv", "report.csv")`, `Problem()`, `ValidationProblem()`.

5.7 DTOs and Mapping

> ✎ **Definition**
>
> **DTOs** (Data Transfer Objects) are simple classes or records shaped for the API contract, kept separate from database entities. **Mapping** converts between the two, by hand or with a library like Mapster.

Never expose EF entities directly: they leak internal columns, cause over-posting vulnerabilities and circular-reference serialisation errors.

```
public record TxnDto(long Id, string Rrn, decimal Amount, string Status);
// Manual mapping (fast, explicit) - or Mapster / AutoMapper / Mapperly (source-generated)
public static class TxnMappings
{
    public static TxnDto ToDto(this Transaction t) => new(t.Id, t.Rrn, t.Amount, t.Status.ToString());
}
var dtos = await db.Transactions.Select(t => new TxnDto(t.Id, t.Rrn, t.Amount, t.Status.ToString()))
    .ToListAsync();
```

> ⚠ **Watch Out**
>
> **Over-posting:** binding directly to an entity (`Update(Transaction txn)`) lets a client set fields like `IsApproved` or `CreatedBy`. Bind to a request DTO that contains only the allowed fields.

5.8 API Versioning and OpenAPI

> ✎ **Definition**
>
> **API versioning** lets several versions of an API coexist (through the URL, query or a header). **OpenAPI** (Swagger) is a machine-readable description of the API, used for documentation and client generation.

```
// Asp.Versioning.Mvc package
builder.Services.AddApiVersioning(o =>
{
    o.DefaultApiVersion = new ApiVersion(1, 0);
    o.AssumeDefaultVersionWhenUnspecified = true;
    o.ApiVersionReader = new UrlSegmentApiVersionReader();
}).AddMvc();
[ApiVersion("1.0")][ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/[controller]")]
// OpenAPI document (.NET 9+): Microsoft.AspNetCore.OpenApi
builder.Services.AddOpenApi();
app.MapOpenApi();                                    // /openapi/v1.json
// UI: Scalar or Swashbuckle UI; .NET 8 templates use AddSwaggerGen() + UseSwagger() + UseSwaggerUI()
```

5.9 JSON Serialization Settings

✎ Definition

Settings for how `System.Text.Json` reads and writes request and response bodies: naming policy, enum handling, null handling, reference cycles and custom converters.

```
builder.Services.AddControllers().AddJsonOptions(o =>
{
    o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter());   // enums as strings
    o.JsonSerializerOptions.DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull;
    o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;
});
```

i Note

ASP.NET Core uses camelCase JSON and case-insensitive deserialization by default (`JsonSerializerDefaults.Web`). Plain `JsonSerializer.Deserialize` outside ASP.NET is **case-sensitive** - a common source of all-null objects.

6. Minimal APIs

6.1 What are Minimal APIs?

> ✎ **Definition**
>
> **Minimal APIs** define HTTP endpoints directly with lambdas (`app.MapGet`, `MapPost`...) without controllers. They are lightweight and fast, and well suited to microservices.

Introduced in .NET 6, minimal APIs define HTTP endpoints with **lambdas or methods directly on the app**, with less ceremony than controllers. They are fast (fewer allocations, Native AOT friendly) and ideal for microservices.

```
var app = WebApplication.Create(args);
app.MapGet("/", () => "Hello Recon!");
app.MapGet("/txns/{id:long}", async (long id, ReconDbContext db) =>
    await db.Transactions.FindAsync(id) is { } t ? Results.Ok(t) : Results.NotFound());
app.MapPost("/txns", async (CreateTxnRequest req, ITxnService svc) =>
{
    var created = await svc.CreateAsync(req);
    return Results.Created($"/txns/{created.Id}", created);
});
app.Run();
```

6.2 Organising Endpoints: Route Groups and TypedResults

> ✎ **Definition**
>
> `MapGroup` creates **route groups** that share a prefix, filters and metadata. `TypedResults` return strongly typed results, which improve OpenAPI output and testability.

```
public static class TxnEndpoints
{
    public static RouteGroupBuilder MapTxnEndpoints(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/txns")
                       .WithTags("Transactions")
                       .RequireAuthorization()
                       .AddEndpointFilter<LoggingFilter>();

        group.MapGet("/", GetAll);
        group.MapGet("/{id:long}", GetById).WithName("GetTxn");
        group.MapPost("/", Create).RequireAuthorization("CanCreateTxn");
        return group;
    }

    static async Task<Ok<List<TxnDto>>> GetAll([AsParameters] TxnQuery q, ITxnService svc,
        CancellationToken ct)
        => TypedResults.Ok(await svc.SearchAsync(q, ct));

    static async Task<Results<Ok<TxnDto>, NotFound>> GetById(long id, ITxnService svc, CancellationToken ct)
        => await svc.GetAsync(id, ct) is { } dto ? TypedResults.Ok(dto) : TypedResults.NotFound();

    static async Task<Results<Created<TxnDto>, ValidationProblem>> Create(CreateTxnRequest req,
        ITxnService svc)
    {
        var created = await svc.CreateAsync(req);
        return TypedResults.Created($"/api/txns/{created.Id}", created);
    }
}
app.MapTxnEndpoints();
```

> ⚡ **Tricky Point**
>
> Prefer `TypedResults` over `Results`: the `Results<T1, T2>` return type documents every possible response in OpenAPI automatically and is checked at compile time.

6.3 Endpoint Filters and Validation

> ✎ **Definition**
>
> **Endpoint filters** (`AddEndpointFilter`) run code before and after a Minimal API handler, for example to validate input or log, much like action filters do for controllers.

```
public class ValidationFilter<T>(IValidator<T> validator) : IEndpointFilter
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext ctx,
        EndpointFilterDelegate next)
    {
        var arg = ctx.Arguments.OfType<T>().First();
        var result = await validator.ValidateAsync(arg);
        if (!result.IsValid) return TypedResults.ValidationProblem(result.ToDictionary());
        return await next(ctx);
    }
}
group.MapPost("/", Create).AddEndpointFilter<ValidationFilter<CreateTxnRequest>>();
// .NET 10: builder.Services.AddValidation(); validates DataAnnotations on minimal API parameters automatically
```

6.4 Controllers vs Minimal APIs

> ✎ **Definition**
>
> A comparison of the two API styles. Controllers suit large, convention-based APIs with filters and model-binding features. Minimal APIs suit small, fast, focused services.

Controllers	Minimal APIs
Class-based, attribute routing, conventions	Lambdas/methods, explicit mapping
Action filters, model validation built in	Endpoint filters; validation built in from .NET 10
Great for large APIs with many conventions	Great for microservices, small APIs, Native AOT
Slightly more overhead	Fastest, smallest
`[ApiController]` features	Route groups, TypedResults

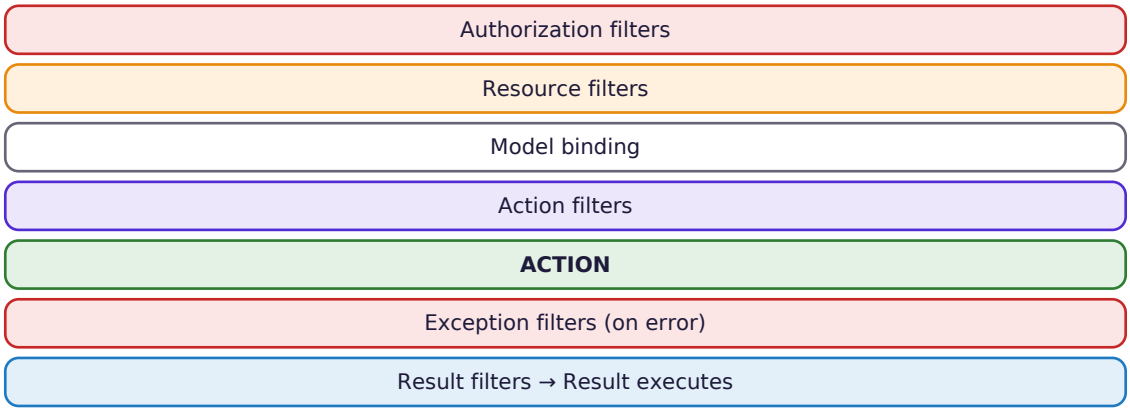
Both run on the same routing, DI, middleware and auth system; you can mix them in one app.

7. Filters and Error Handling

7.1 MVC Filter Pipeline

> ✎ **Definition**
>
> **Filters** run code at specific stages of MVC action execution: Authorization, Resource, Action, Exception and Result filters, each wrapping the next stage.

Filters run code **before or after specific stages** of MVC action processing (after routing has selected a controller action).



Filters run inside MVC after routing; middleware runs for every request before it.

Figure 7.1 - Order of filter execution

Filter	Interface	Typical use
Authorization	`IAuthorizationFilter`	Custom auth checks (prefer policies)
Resource	`IResourceFilter`	Caching, short-circuit before model binding
Action	`IActionFilter` / `IAsyncActionFilter`	Logging, validation, modifying arguments/results
Exception	`IExceptionFilter`	Handle exceptions from actions
Result	`IResultFilter`	Modify the response (headers, wrapping)

```
public class AuditFilter(ILogger<AuditFilter> logger) : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(ActionExecutingContext ctx, ActionExecutionDelegate next)
    {
        var user = ctx.HttpContext.User.Identity?.Name ?? "anonymous";
        logger.LogInformation("{User} calling {Action}", user, ctx.ActionDescriptor.DisplayName);
        var executed = await next();                                    // runs the action
        if (executed.Exception is not null) logger.LogWarning("Action failed");
    }
}
// Apply: globally, per controller, or per action
builder.Services.AddControllers(o => o.Filters.Add<AuditFilter>());
[ServiceFilter(typeof(AuditFilter))]   // resolved from DI (register it)
[TypeFilter(typeof(AuditFilter))]      // created by DI activation, no registration needed
```

★ Interview Tip

"Middleware vs filters?" - Middleware runs for **every request** and knows only `HttpContext`. Filters run only for MVC/endpoint actions and have access to action context (arguments, model state, the selected action, the result). Use middleware for cross-cutting HTTP concerns; filters for MVC-specific concerns.

7.2 Global Error Handling with ProblemDetails

✎ Definition

Handling unhandled exceptions in one place (`UseExceptionHandler` or `IExceptionHandler`) and returning a standard **ProblemDetails** (RFC 7807/9457) JSON error response.

The standard error format for HTTP APIs is **RFC 9457 Problem Details** (`application/problem+json`).

```
builder.Services.AddProblemDetails(o => o.CustomizeProblemDetails = c =>
    c.ProblemDetails.Extensions["traceId"] = c.HttpContext.TraceIdentifier);
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();          // .NET 8 IExceptionHandler

app.UseExceptionHandler();
app.UseStatusCodePages();                                                 // ProblemDetails for 404/405 etc.

public class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger, IProblemDetailsService pds)
    : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, CancellationToken ct)
    {
        var (status, title) = ex switch
        {
            NotFoundException => (StatusCodes.Status404NotFound, "Resource not found"),
            ValidationException => (StatusCodes.Status400BadRequest, "Validation failed"),
            DuplicateTxnException => (StatusCodes.Status409Conflict, "Duplicate transaction"),
            _ => (StatusCodes.Status500InternalServerError, "An unexpected error occurred")
        };
        if (status == 500) logger.LogError(ex, "Unhandled exception");
        ctx.Response.StatusCode = status;
        return await pds.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = ctx,
            ProblemDetails = { Title = title, Status = status, Detail = status < 500 ? ex.Message : null },
            Exception = ex
        });
    }
}
```

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "Duplicate transaction",
  "status": 409,
  "detail": "RRN 123456789012 already exists",
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
}
```

⚠ Watch Out

Never return `ex.Message`/stack traces of unexpected 500 errors to clients in production - they leak internals (SQL, paths, connection details). Log the details with a trace ID, return a generic message plus the trace ID.

7.3 Exceptions vs Result Objects

> ✎ **Definition**
>
> Two ways to report failures: **throwing exceptions** for unexpected errors, or returning a **Result object** (success or error value) for expected business failures, without the cost of exceptions.

Exceptions are expensive and best for **exceptional** situations. For expected business failures (validation, not found, insufficient balance) many teams return a **Result** type.

```
public record Result<T>(T? Value, string? Error)
{
    public bool IsSuccess => Error is null;
    public static Result<T> Ok(T v) => new(v, null);
    public static Result<T> Fail(string e) => new(default, e);
}
var result = await service.TransferAsync(cmd);
return result.IsSuccess ? Ok(result.Value) : BadRequest(result.Error);
```

8. Entity Framework Core

8.1 What is EF Core?

> ✎ **Definition**
>
> **Entity Framework Core** is .NET's **ORM** (Object-Relational Mapper). It maps C# classes to database tables and lets you query and save data with LINQ through a `DbContext`.

EF Core is Microsoft's **ORM (Object-Relational Mapper)**: you work with C# classes and LINQ, and EF Core generates SQL, tracks changes and maps results back to objects. It supports SQL Server, PostgreSQL, MySQL, SQLite, Oracle, Cosmos DB and more.

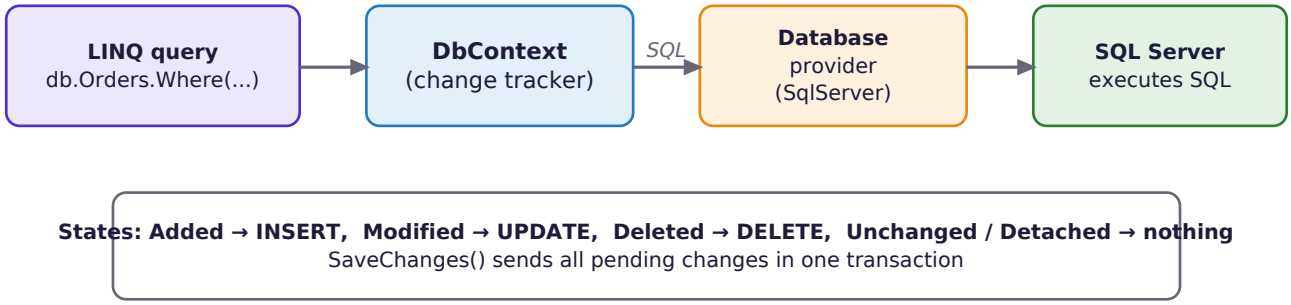


Figure 8.1 - How EF Core turns LINQ into SQL

```
public class Transaction
{
    public long Id { get; set; }                                    // PK by convention
    public string Rrn { get; set; } = "";
    public decimal Amount { get; set; }
    public TxnStatus Status { get; set; }
    public DateTime CreatedUtc { get; set; }
    public int BankId { get; set; }                                 // FK by convention
    public Bank Bank { get; set; } = null!;                         // navigation (many-to-one)
    public byte[] RowVersion { get; set; } = [];                    // concurrency token
}
public class Bank
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public List<Transaction> Transactions { get; set; } = [];       // one-to-many
}

public class ReconDbContext(DbContextOptions<ReconDbContext> options) : DbContext(options)
{
    public DbSet<Transaction> Transactions => Set<Transaction>();
    public DbSet<Bank> Banks => Set<Bank>();

    protected override void OnModelCreating(ModelBuilder mb)
    {
        mb.ApplyConfigurationsFromAssembly(typeof(ReconDbContext).Assembly);
    }
}

public class TransactionConfig : IEntityTypeConfiguration<Transaction>      // Fluent API
{
    public void Configure(EntityTypeBuilder<Transaction> b)
    {
        b.ToTable("Transactions");
        b.Property(t => t.Rrn).HasMaxLength(12).IsRequired();
        b.HasIndex(t => t.Rrn).IsUnique();
        b.Property(t => t.Amount).HasPrecision(18, 2);              // always set decimal precision!
        b.Property(t => t.Status).HasConversion<string>().HasMaxLength(20);
        b.Property(t => t.RowVersion).IsRowVersion();
        b.HasOne(t => t.Bank).WithMany(bk => bk.Transactions).HasForeignKey(t => t.BankId)
```

```
         .OnDelete(DeleteBehavior.Restrict);
        b.HasQueryFilter(t => t.Status != TxnStatus.Deleted);       // global filter (soft delete)
    }
}
builder.Services.AddDbContext<ReconDbContext>(o => o
    .UseSqlServer(builder.Configuration.GetConnectionString("Recon"),
                  sql => sql.EnableRetryOnFailure().CommandTimeout(60)));
```

⚡ Tricky Point

Without `HasPrecision(18, 2)` (or `[Precision]`), EF Core warns and SQL Server defaults `decimal` to (18,2) silently; other providers may truncate. For money, always configure precision explicitly.

8.2 Migrations (Code First)

✎ Definition

Migrations keep the database schema in sync with the C# model (Code First): `dotnet ef migrations add` creates a schema change script, and `database update` applies it.

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate             // generates a migration class from model changes
dotnet ef database update                           // applies pending migrations
dotnet ef migrations remove                         // remove the last (unapplied) migration
dotnet ef migrations script --idempotent -o deploy.sql   // SQL script for DBAs / production
dotnet ef dbcontext scaffold "Server=..." Microsoft.EntityFrameworkCore.SqlServer -o Models   // DB first
```

⚠ Watch Out

Calling `db.Database.Migrate()` at app start-up is convenient but risky in production with multiple instances (concurrent migrations) and needs DDL permissions. Prefer idempotent SQL scripts or migration bundles (`dotnet ef migrations bundle`) in your CI/CD pipeline.

8.3 Querying

✎ Definition

Reading data with LINQ against DbSet<T>, which EF Core translates to SQL. Key tools are filtering, projection, AsNoTracking, pagination, and raw SQL when needed.

```
var failed = await db.Transactions
    .AsNoTracking()                                  // read-only: faster, no change tracking
    .Where(t => t.Status == TxnStatus.Failed && t.CreatedUtc >= from)
    .OrderByDescending(t => t.CreatedUtc)
    .Skip((page - 1) * size).Take(size)              // paging -> OFFSET/FETCH
    .Select(t => new TxnDto(t.Id, t.Rrn, t.Amount, t.Bank.Name))   // projection: only needed columns
    .ToListAsync(ct);

var txn = await db.Transactions.FindAsync([id], ct);             // checks the tracker first, then DB
var withBank = await db.Transactions.Include(t => t.Bank).FirstOrDefaultAsync(t => t.Id == id, ct);
// filtered include
var banks = await db.Banks.Include(b => b.Transactions.Where(t => t.Amount > 10000)).ToListAsync(ct);
bool exists = await db.Transactions.AnyAsync(t => t.Rrn == rrn, ct);
int count = await db.Transactions.CountAsync(ct);
var totals = await db.Transactions.GroupBy(t => t.BankId)
                    .Select(g => new { BankId = g.Key, Total = g.Sum(t => t.Amount) }).ToListAsync(ct);
// parameterised
var raw = await db.Transactions.FromSql($"SELECT * FROM Transactions WHERE Rrn = {rrn}").ToListAsync();
// .NET 8 unmapped types
var dtos = await db.Database.SqlQuery<TxnSummary>($"EXEC dbo.usp_GetSummary {date}").ToListAsync();
```

⚡ Tricky Point

`FromSql($"... {rrn}")` and `SqlQuery($"...")` **parameterise** interpolated values (safe). `FromSqlRaw("... " + rrn)` with concatenation is **SQL injection**. Never build SQL strings from user input.

IQueryable vs IEnumerable (again, because it matters)

```
IQueryable<Transaction> q = db.Transactions.Where(t => t.Amount > 1000);   // SQL WHERE
var list = q.ToList();                                                     // executes now
// loads ENTIRE table, filters in memory
var bad = db.Transactions.AsEnumerable().Where(t => t.Amount > 1000);
```

8.4 Loading Related Data

✎ Definition

Strategies for loading navigation properties: **eager** (`Include`), **explicit** (`Entry().Load`), **lazy** (proxies, loaded on access), and **split queries** to avoid cartesian explosion.

Strategy	How	Pros/cons
Eager loading	`Include()` / `ThenInclude()`	One query (JOIN); risk of cartesian explosion with many collections
Split queries	`.AsSplitQuery()`	Separate SQL per collection; avoids duplication
Explicit loading	`db.Entry(bank).Collection(b => b.Transactions).LoadAsync()`	Load on demand, deliberately
Lazy loading	Proxies + `virtual` navigations	Convenient but causes **N+1 queries**
Projection	`Select(...)`	Best for read APIs: exactly the needed columns

⚡ Tricky Point

N+1 problem: loading 100 banks and then touching `bank.Transactions` in a loop with lazy loading fires **101 queries**. Use `Include`, projection, or batch queries. Turn on SQL logging (`LogTo(Console.WriteLine)` or `EnableSensitiveDataLogging` in dev) to see it.

8.5 Saving Data and Change Tracking

✎ Definition

The **change tracker** records the state of loaded entities (Added, Modified, Deleted...). `SaveChanges()` turns those changes into INSERT, UPDATE and DELETE statements in one transaction.

```
db.Transactions.Add(new Transaction { Rrn = "123456789012", Amount = 500, BankId = 1 });   // Added
var t = await db.Transactions.FirstAsync(x => x.Id == id);
t.Status = TxnStatus.Matched;                                          // tracker detects -> Modified
db.Transactions.Remove(other);                                        // Deleted
await db.SaveChangesAsync(ct);                                         // one transaction, all changes

// Bulk operations without loading entities (EF Core 7+)
await db.Transactions.Where(x => x.CreatedUtc < cutoff).ExecuteDeleteAsync(ct);
await db.Transactions.Where(x => x.Status == TxnStatus.Pending && x.CreatedUtc < cutoff)
        .ExecuteUpdateAsync(s => s.SetProperty(x => x.Status, TxnStatus.Expired), ct);
```

Entity state	Meaning	SaveChanges does
Added	New, not in DB	INSERT
Unchanged	Loaded, not modified	Nothing
Modified	Property values changed	UPDATE (changed columns)
Deleted	Marked for removal	DELETE
Detached	Not tracked	Nothing

> ⚡ **Tricky Point**
>
> `ExecuteUpdate`/`ExecuteDelete` run **immediately** as SQL and **bypass** the change tracker, `SaveChanges` interceptors and in-memory entities - tracked objects can become stale.

8.6 Transactions and Concurrency

> ✎ **Definition**
>
> **Transactions** group several operations so they succeed or fail together. **Concurrency control** (optimistic, using a row version or concurrency token) detects conflicting updates by different users.

```
await using var tx = await db.Database.BeginTransactionAsync(ct);
try
{
    db.Settlements.Add(settlement);
    await db.SaveChangesAsync(ct);
    await db.Transactions.Where(t => t.BatchId == batchId)
            .ExecuteUpdateAsync(s => s.SetProperty(t => t.Status, TxnStatus.Settled), ct);
    await tx.CommitAsync(ct);
}
catch { await tx.RollbackAsync(ct); throw; }
// Optimistic concurrency with rowversion
try { await db.SaveChangesAsync(ct); }
catch (DbUpdateConcurrencyException) { return Conflict("Record was modified by another user"); }
```

> ⚠ **Watch Out**
>
> With `EnableRetryOnFailure`, user-initiated transactions must run inside `db.Database.CreateExecutionStrategy().ExecuteAsync(...)`, otherwise EF throws because it cannot retry a partially executed transaction.

8.7 DbContext Rules and Performance

> ✎ **Definition**
>
> Best practices for `DbContext`: it is **scoped and not thread-safe**. Use `AsNoTracking` for reads, project only the needed columns, avoid N+1 queries, and consider pooling and compiled queries.

- `DbContext` is **not thread-safe** and is designed as a **short-lived unit of work** - register it **scoped** (the default with `AddDbContext`).
- Never run parallel queries on the same context (`Task.WhenAll` of two queries throws). Use `IDbContextFactory<T>` to create separate contexts.
- In background services/singletons, create a scope or use the factory per unit of work.
- `AsNoTracking()` for read-only queries; projections for APIs; `AddDbContextPool` to reuse context instances.
- Add indexes for filter/join columns; check generated SQL; use compiled queries for hot paths.
- For huge inserts (millions of rows) use `SqlBulkCopy` or EFCore.BulkExtensions instead of `AddRange` + `SaveChanges`.
- Use `CancellationToken` on every async call so abandoned requests stop their queries.

★ Interview Tip

"EF Core vs Dapper?" - EF Core provides change tracking, migrations, LINQ and relationships (productive, good for most CRUD). Dapper is a micro-ORM that maps raw SQL results to objects (maximum control and speed for complex reporting queries and stored procedures). Many real systems use both.

```
using var conn = new SqlConnection(cs);
var rows = await conn.QueryAsync<TxnDto>(
    "SELECT Id, Rrn, Amount FROM Transactions WHERE BankId = @bankId", new { bankId });
var summary = await conn.QueryAsync<Summary>("dbo.usp_ReconSummary",
    new { Date = date }, commandType: CommandType.StoredProcedure);
```

9. Authentication, Authorization and Security

9.1 Authentication vs Authorization

> ✎ **Definition**
>
> **Authentication** establishes *who* the user is (identity). **Authorization** decides *what* that authenticated user is allowed to do (permissions).

Authentication	Authorization
"Who are you?"	"What are you allowed to do?"
Validates credentials/tokens, builds `HttpContext.User` (ClaimsPrincipal)	Checks roles, claims, policies, resources
`UseAuthentication()`	`UseAuthorization()`, `[Authorize]`
Failure: 401 Unauthorized	Failure: 403 Forbidden

A user is a **ClaimsPrincipal** containing one or more **ClaimsIdentity** objects, each with **claims** (key-value facts: name, email, role, bankId).

9.2 JWT Bearer Authentication

> ✎ **Definition**
>
> **JWT** (JSON Web Token) is a signed token containing user claims. With **bearer authentication**, the client sends it in the `Authorization: Bearer` header and the API validates it on every request.

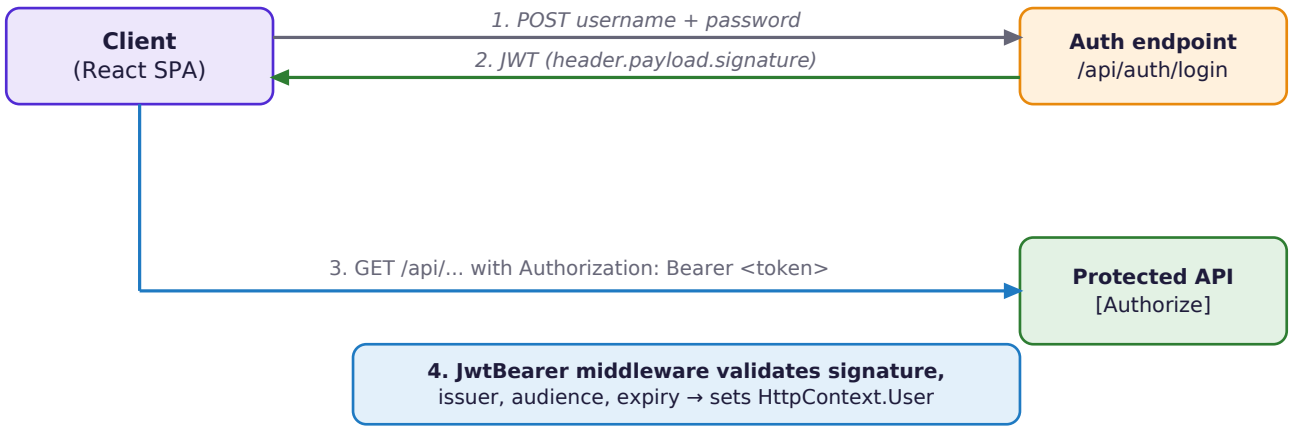


Figure 9.1 - JWT login and API call flow

A **JWT (JSON Web Token)** has three Base64Url parts: `header.payload.signature`. The payload holds claims (`sub`, `exp`, `role`...) - it is **signed, not encrypted**, so anyone can read it.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true, ValidIssuer = cfg["Jwt:Issuer"],
            ValidateAudience = true, ValidAudience = cfg["Jwt:Audience"],
            ValidateLifetime = true, ClockSkew = TimeSpan.FromMinutes(1),   // default skew is 5 minutes!
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(cfg["Jwt:Key"]!))
        };
    });
builder.Services.AddAuthorization();

// Issuing a token (login endpoint)
string CreateToken(AppUser user, IList<string> roles)
{
    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, user.Id),
        new(JwtRegisteredClaimNames.Email, user.Email),
        new("bank_id", user.BankId.ToString()),
    };
    claims.AddRange(roles.Select(r => new Claim(ClaimTypes.Role, r)));
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(cfg["Jwt:Key"]!));   // >= 256-bit secret
    var token = new JwtSecurityToken(cfg["Jwt:Issuer"], cfg["Jwt:Audience"], claims,
        expires: DateTime.UtcNow.AddMinutes(15),
        signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256));
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

⚡ Tricky Point

Never put secrets or sensitive data in a JWT payload - it's only Base64 encoded. Keep access tokens short-lived (5-15 minutes) and use **refresh tokens** (stored server-side, rotated, revocable) to get new ones. A JWT can't be revoked before expiry unless you add a deny-list.

⚡ Tricky Point

`ClockSkew` defaults to **5 minutes**: a token "expiring in 1 minute" in your test still works for 6 minutes. Set it lower if you test expiry.

9.3 Authorization: Roles, Claims and Policies

✎ Definition

Ways to control access: **role-based** (`[Authorize(Roles="Admin")]`), **claims-based**, and **policy-based** authorization, where named policies combine requirements and handlers.

```
[Authorize]                                  // any authenticated user
[Authorize(Roles = "Admin,Supervisor")]      // either role
[AllowAnonymous]                             // override on an action
[Authorize(Policy = "SameBankOnly")]

builder.Services.AddAuthorizationBuilder()
    .AddPolicy("CanApproveRefund", p => p.RequireRole("Supervisor").RequireClaim("dept", "Ops"))
    .AddPolicy("HighValue", p => p.RequireAssertion(ctx =>
        ctx.User.HasClaim(c => c.Type == "limit" && decimal.Parse(c.Value) >= 100000)))
    // secure by default
    .SetFallbackPolicy(new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build());
```

Custom requirement and resource-based authorization

```
public class SameBankRequirement : IAuthorizationRequirement { }
public class SameBankHandler : AuthorizationHandler<SameBankRequirement, Transaction>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext ctx,
        SameBankRequirement req, Transaction txn)
    {
        if (ctx.User.FindFirst("bank_id")?.Value == txn.BankId.ToString()) ctx.Succeed(req);
        return Task.CompletedTask;
    }
}
builder.Services.AddSingleton<IAuthorizationHandler, SameBankHandler>();

// In the controller, after loading the resource:
var result = await _authz.AuthorizeAsync(User, txn, new SameBankRequirement());
if (!result.Succeeded) return Forbid();
```

★ Interview Tip

"Roles vs claims vs policies?" - Roles are coarse groups. Claims are facts about the user. Policies combine requirements (roles, claims, custom logic) into a named rule, keeping authorization logic out of controllers and testable.

9.4 ASP.NET Core Identity

✎ Definition

ASP.NET Core Identity is a membership system for users, passwords, roles, claims, tokens, lockout and two-factor authentication, stored through EF Core.

Identity is a membership system: users, password hashing (PBKDF2), roles, claims, lockout, 2FA, email confirmation, external logins.

```
builder.Services.AddIdentityCore<AppUser>(o =>
    {
        o.Password.RequiredLength = 10;
        o.Lockout.MaxFailedAccessAttempts = 5;
    })
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ReconDbContext>();
// .NET 8: ready-made API endpoints
builder.Services.AddIdentityApiEndpoints<AppUser>().AddEntityFrameworkStores<ReconDbContext>();
app.MapIdentityApi<AppUser>();       // /register, /login, /refresh, /manage/2fa ...
```

For enterprise SSO use **OpenID Connect** with an identity provider: Microsoft Entra ID (Azure AD), Keycloak, Auth0, Okta, Duende IdentityServer.

9.5 CORS

✎ Definition

CORS (Cross-Origin Resource Sharing) is a browser security mechanism. The server sends headers that say which other origins (domains) may call the API from JavaScript.

Browsers block JavaScript from calling an API on a **different origin** (scheme + host + port) unless the API allows it with CORS headers.

```
builder.Services.AddCors(o => o.AddPolicy("Spa", p => p
    .WithOrigins("https://recon.mycompany.com", "http://localhost:5173")
    .AllowAnyHeader().AllowAnyMethod().AllowCredentials()));
app.UseCors("Spa");      // after UseRouting, before UseAuthentication/UseAuthorization
```

⚡ Tricky Point

`AllowAnyOrigin()` cannot be combined with `AllowCredentials()` (the browser rejects `*` with credentials). CORS is enforced by **browsers only** - it is not a security boundary against Postman, curl or servers.

9.6 Rate Limiting (.NET 7+)

✎ Definition

Rate limiting restricts how many requests a client may make in a time window (fixed window, sliding window, token bucket, concurrency) to protect the API from abuse. Built in via `AddRateLimiter`.

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    o.AddFixedWindowLimiter("fixed", l => { l.PermitLimit = 100; l.Window = TimeSpan.FromMinutes(1); });
    o.AddPolicy("per-user", ctx => RateLimitPartition.GetTokenBucketLimiter(
        ctx.User.Identity?.Name ?? ctx.Connection.RemoteIpAddress?.ToString() ?? "anon",
        _ => new TokenBucketRateLimiterOptions { TokenLimit = 20, TokensPerPeriod = 10,
                                                 ReplenishmentPeriod = TimeSpan.FromSeconds(10) }));
});
app.UseRateLimiter();
app.MapPost("/api/login", Login).RequireRateLimiting("fixed");
```

9.7 Security Checklist

✎ Definition

A list of essential security practices: HTTPS and HSTS, input validation, parameterised queries, secret management, least privilege, security headers, and dependency updates.

- HTTPS everywhere (`UseHttpsRedirection`, `UseHsts`), TLS 1.2+.
- Parameterised queries only (EF Core/Dapper parameters); never concatenate SQL.
- Validate all input (DTOs, FluentValidation); limit request/upload sizes.
- Secrets in Key Vault/environment, not source control; rotate keys.
- Hash passwords with Identity's hasher (never store plain or reversible).
- Data Protection API for encrypting cookies/tokens (persist keys in a shared store for web farms).
- Anti-forgery tokens for cookie-authenticated forms (CSRF); SameSite cookies.
- Security headers: Content-Security-Policy, X-Content-Type-Options, X-Frame-Options.
- Don't expose stack traces; generic errors with trace IDs.
- Least-privilege DB accounts; audit logs for sensitive operations.
- For payment data (PCI DSS): never log full card numbers (PAN) or CVV; mask PANs (first 6 / last 4); encrypt data at rest.

10. Caching and Performance

10.1 Caching Options

> ✎ **Definition**
>
> Ways to store data so it can be served faster: **in-memory** (`IMemoryCache`), **distributed** (`IDistributedCache`, e.g. Redis), **HybridCache**, **output caching** and **response caching**.



HybridCache (.NET 9) combines L1 memory + L2 distributed and prevents cache stampedes.

Figure 10.1 - Layered caching

Cache	Where	Use
`IMemoryCache`	In-process memory	Single server, small reference data
`IDistributedCache`	Redis, SQL Server, NCache	Web farms, data shared by instances
`HybridCache` (.NET 9)	L1 memory + L2 distributed	Best default: stampede protection, tags
Output caching (.NET 7)	Server-side response cache	Cache whole endpoint responses
Response caching	HTTP headers (`Cache-Control`)	Let browsers/proxies cache

```
// IMemoryCache
public class BankService(IMemoryCache cache, ReconDbContext db)
{
    public Task<List<Bank>> GetBanksAsync() =>
        cache.GetOrCreateAsync("banks:all", async e =>
        {
            e.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30);
            e.SlidingExpiration = TimeSpan.FromMinutes(5);
            return await db.Banks.AsNoTracking().ToListAsync();
        })!;
}

// HybridCache (.NET 9)
builder.Services.AddHybridCache();
public async Task<TxnSummary> GetSummaryAsync(DateOnly date, CancellationToken ct) =>
    await _cache.GetOrCreateAsync($"summary:{date}",
        async token => await _repo.BuildSummaryAsync(date, token),
        tags: ["summaries"], cancellationToken: ct);
await _cache.RemoveByTagAsync("summaries");     // invalidate a group
// Output cache
builder.Services.AddOutputCache(o => o.AddPolicy("Short", b => b.Expire(TimeSpan.FromSeconds(30))));
app.UseOutputCache();
app.MapGet("/api/banks", GetBanks).CacheOutput("Short");
```

> ⚡ **Tricky Point**
>
> **Cache stampede:** when a popular key expires, hundreds of concurrent requests miss at once and all hit the database. `IMemoryCache.GetOrCreateAsync` does **not** prevent this; `HybridCache` does (only one caller executes the factory). Also remember **cache invalidation** when the underlying data changes.

10.2 Async All the Way

> ✎ **Definition**
>
> Using `async`/`await` through every layer for I/O work, so request threads are freed while waiting and the server can handle many more requests. Avoid `.Result`/`.Wait()`.

ASP.NET Core serves requests from the **thread pool**. Blocking a thread while waiting for I/O wastes it; under load the pool starves and latency explodes.

```
public async Task<IActionResult> Get(CancellationToken ct)
{
    var data = await _repo.GetAsync(ct);          // GOOD: thread returns to pool while waiting
    return Ok(data);
}
var bad = _repo.GetAsync().Result;                 // BAD: sync-over-async, thread blocked
```

> ⚠ **Watch Out**
>
> **Thread-pool starvation** symptoms: CPU low, but requests slow and timing out under load. Common causes: `.Result`/`.Wait()`, `Thread.Sleep`, synchronous file/DB/HTTP APIs in request paths.

10.3 Performance Checklist

> ✎ **Definition**
>
> A list of techniques for fast APIs: async I/O, caching, efficient EF queries, pagination, response compression, pooling, avoiding large allocations, and profiling before optimising.

- Use `AsNoTracking` and projections in EF Core; avoid N+1; add DB indexes.
- Paginate list endpoints; never return unbounded result sets.
- Response compression (`AddResponseCompression`), especially for large JSON.
- Reuse `HttpClient` via `IHttpClientFactory` (Chapter 11).
- Cache reference data; use `HybridCache`/Redis in web farms.
- Stream large files and results (`IAsyncEnumerable<T>`, `FileStreamResult`) instead of buffering.
- Avoid large object allocations in hot paths; use `ArrayPool<T>`, `Span<T>`, `StringBuilder`.
- Use Server GC (default in ASP.NET Core) and consider `DOTNET_TieredPGO` (on by default since .NET 8).
- Measure: BenchmarkDotNet for code, `dotnet-counters` / `dotnet-trace` for running apps, load tests with k6 or NBomber, APM (Application Insights, OpenTelemetry).
- Consider Native AOT for fast start-up microservices.

11. Background Services, HttpClient and Resilience

11.1 Hosted Services and BackgroundService

> ✎ **Definition**
>
> A **hosted service** (`IHostedService`) runs background work alongside the web app. `BackgroundService` is a base class for long-running loops, via `ExecuteAsync`.

`IHostedService` runs code alongside the app (start/stop with the host). `BackgroundService` is a convenient base class with a long-running `ExecuteAsync`.

```
public class ReconWorker(IServiceScopeFactory scopeFactory, ILogger<ReconWorker> logger,
                         IOptionsMonitor<ReconOptions> options) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(5));
        do
        {
            try
            {
                using var scope = scopeFactory.CreateScope();          // NEW scope per run
                var recon = scope.ServiceProvider.GetRequiredService<IReconService>();   // scoped service
                int matched = await recon.RunAsync(DateOnly.FromDateTime(DateTime.UtcNow), stoppingToken);
                logger.LogInformation("Matched {Count}", matched);
            }
            catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested) { break; }
            catch (Exception ex)
            {
                // don't let one failure kill the worker
                logger.LogError(ex, "Recon run failed");
            }
        }
        while (await timer.WaitForNextTickAsync(stoppingToken));
    }
}
builder.Services.AddHostedService<ReconWorker>();
```

> ⚡ **Tricky Point**
>
> Hosted services are **singletons**, so you cannot inject `DbContext` or other scoped services into their constructor. Inject `IServiceScopeFactory` and create a scope per unit of work.

> ⚡ **Tricky Point**
>
> Since .NET 6, an **unhandled exception in `ExecuteAsync` stops the whole host** by default (`BackgroundServiceExceptionBehavior.StopHost`). Wrap the loop body in try/catch. Also, a long synchronous block before the first `await` delays application start-up - start with `await Task.Yield()` if needed.

Worker Service project (Windows Service / Linux daemon)

```
dotnet new worker -n Recon.Worker
var builder = Host.CreateApplicationBuilder(args);
// Microsoft.Extensions.Hosting.WindowsServices
builder.Services.AddWindowsService(o => o.ServiceName = "Recon Worker");
builder.Services.AddSystemd();                                          // Linux
builder.Services.AddHostedService<ReconWorker>();
builder.Build().Run();
// install: sc.exe create "Recon Worker" binPath= "C:\apps\Recon.Worker.exe"
```

Background job libraries

Need	Use
Simple periodic work	`BackgroundService` + `PeriodicTimer`
Queue of in-process work items	`Channel<T>` + `BackgroundService`
Persistent jobs, retries, dashboard, cron	Hangfire, Quartz.NET
Distributed messaging between services	RabbitMQ / Azure Service Bus / Kafka with MassTransit or NServiceBus

```
// In-process queue with Channel<T>
public class FileQueue
{
    public Channel<string> Channel { get; } = System.Threading.Channels.Channel.CreateBounded<string>(100);
}
builder.Services.AddSingleton<FileQueue>();
// producer (controller): await queue.Channel.Writer.WriteAsync(path, ct);
// consumer (BackgroundService):
await foreach (var path in queue.Channel.Reader.ReadAllAsync(stoppingToken)) await ProcessAsync(path);
```

> **⚠ Watch Out**
>
> Fire-and-forget work started from a request (`_ = Task.Run(...)`) is lost if the app restarts, and scoped services captured from the request are disposed when the request ends (`ObjectDisposedException`). Hand work to a queue/background service instead.

11.2 IHttpClientFactory

> **✎ Definition**
>
> `IHttpClientFactory` creates and manages `HttpClient` instances, pooling handlers to avoid socket exhaustion and stale DNS. It supports named and typed clients.

Creating `new HttpClient()` per request exhausts **sockets** (TIME_WAIT); a single static `HttpClient` doesn't respect **DNS changes**. `IHttpClientFactory` pools and recycles `HttpMessageHandler`s, solving both.

```
// Typed client (recommended)
builder.Services.AddHttpClient<INpciClient, NpciClient>(c =>
{
    c.BaseAddress = new Uri(builder.Configuration["Npci:BaseUrl"]!);
    c.Timeout = TimeSpan.FromSeconds(30);
    c.DefaultRequestHeaders.Add("X-Client-Id", "recon");
});

public class NpciClient(HttpClient http) : INpciClient
{
    public async Task<SettlementFile?> GetSettlementAsync(DateOnly date, CancellationToken ct)
    {
        using var res = await http.GetAsync($"settlements/{date:yyyyMMdd}", ct);
        if (res.StatusCode == HttpStatusCode.NotFound) return null;
        res.EnsureSuccessStatusCode();
        return await res.Content.ReadFromJsonAsync<SettlementFile>(ct);
    }
}

// Named client
builder.Services.AddHttpClient("bank", c => c.BaseAddress = new Uri("https://api.bank.com/"));
var client = httpClientFactory.CreateClient("bank");

// Message handler for auth headers
public class BearerTokenHandler(ITokenProvider tokens) : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage req, CancellationToken ct)
    {
        req.Headers.Authorization = new AuthenticationHeaderValue("Bearer", await tokens.GetAsync(ct));
        return await base.SendAsync(req, ct);
    }
}
builder.Services.AddTransient<BearerTokenHandler>();
builder.Services.AddHttpClient<INpciClient, NpciClient>().AddHttpMessageHandler<BearerTokenHandler>();
```

> ⚡ **Tricky Point**
>
> Typed clients are registered as **transient**. Don't inject a typed client into a **singleton** - it captures one handler forever and loses DNS refresh. In singletons, inject `IHttpClientFactory` and call `CreateClient` when needed.

11.3 Resilience with Polly (Microsoft.Extensions.Http.Resilience)

> ✎ **Definition**
>
> **Resilience** means handling transient failures in calls to other services with strategies such as retry, timeout, circuit breaker and fallback, using Polly via **Microsoft.Extensions.Http.Resilience**.

Remote calls fail transiently (timeouts, 503s, network blips). Resilience strategies make the app tolerate them.

```
builder.Services.AddHttpClient<INpciClient, NpciClient>()
    .AddStandardResilienceHandler();      // rate limiter, total timeout, retry (exp. backoff + jitter),
                                          // circuit breaker, attempt timeout - sensible defaults (.NET 8)

builder.Services.AddHttpClient<IBankClient, BankClient>()
    .AddResilienceHandler("bank", p =>
    {
        p.AddRetry(new HttpRetryStrategyOptions { MaxRetryAttempts = 3,
            BackoffType = DelayBackoffType.Exponential, UseJitter = true });
        p.AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions { FailureRatio = 0.5,
            BreakDuration = TimeSpan.FromSeconds(30) });
        p.AddTimeout(TimeSpan.FromSeconds(10));
    });
```

Strategy	Purpose
Retry	Try again after transient failures (with exponential backoff + jitter)
Circuit breaker	Stop calling a failing service for a while; fail fast
Timeout	Don't wait forever
Fallback	Return a default/cached response
Rate limiter / bulkhead	Limit concurrent calls
Hedging	Send a parallel request if the first is slow

> ⚠ **Watch Out**
>
> Only retry **idempotent** operations (GET, PUT, DELETE) or requests with an idempotency key. Retrying a non-idempotent POST like "debit account" can charge a customer twice.

12. Real-Time, gRPC, Health Checks and More

12.1 SignalR (Real-Time Communication)

> **✎ Definition**
>
> **SignalR** is a library for **real-time**, two-way communication between server and clients (chat, notifications, live dashboards). It uses WebSockets with automatic fallbacks.

SignalR pushes messages from server to clients in real time over WebSockets (falling back to Server-Sent Events or long polling).

```
public class ReconHub : Hub
{
    public async Task JoinBank(string bankId) => await Groups.AddToGroupAsync(Context.ConnectionId, bankId);
    public override Task OnConnectedAsync() { /* ... */ return base.OnConnectedAsync(); }
}
builder.Services.AddSignalR();
app.MapHub<ReconHub>("/hubs/recon");

// Push from any service
public class ReconNotifier(IHubContext<ReconHub> hub)
{
    public Task BatchCompleted(string bankId, int matched) =>
        hub.Clients.Group(bankId).SendAsync("batchCompleted", new { bankId, matched });
}
```

```
// React client (@microsoft/signalr)
const conn = new HubConnectionBuilder().withUrl('/hubs/recon', { accessTokenFactory: () => token })
  .withAutomaticReconnect().build();
conn.on('batchCompleted', data => toast(`Matched ${data.matched}`));
await conn.start(); await conn.invoke('JoinBank', 'HDFC');
```

> **i Note**
>
> With multiple server instances, use a **backplane** (Redis or Azure SignalR Service) so a message sent on server A reaches clients connected to server B. Load balancers need sticky sessions unless WebSockets-only with skip negotiation.

12.2 gRPC

> **✎ Definition**
>
> **gRPC** is a high-performance RPC framework that uses HTTP/2 and Protocol Buffers (binary, contract-first `.proto` files). It suits fast service-to-service communication.

High-performance RPC using **HTTP/2** and **Protocol Buffers** (binary, strongly typed contracts). Ideal for internal service-to-service communication.

```
// recon.proto
syntax = "proto3";
service ReconService { rpc GetStatus (StatusRequest) returns (StatusReply); }
message StatusRequest { string batch_id = 1; }
message StatusReply { string status = 1; int32 matched = 2; }

public class ReconGrpcService : ReconService.ReconServiceBase
{
    public override Task<StatusReply> GetStatus(StatusRequest req, ServerCallContext ctx) =>
        Task.FromResult(new StatusReply { Status = "DONE", Matched = 1500 });
}
builder.Services.AddGrpc(); app.MapGrpcService<ReconGrpcService>();
```

REST/JSON	gRPC
Human-readable, browser-friendly	Binary, smaller and faster
HTTP/1.1 or 2	HTTP/2 (streaming in both directions)

REST/JSON	gRPC
Contract optional (OpenAPI)	Contract-first (.proto), code generation
Public APIs	Internal microservices

12.3 Health Checks

✎ Definition

Health checks expose endpoints (e.g. `/health`) that report whether the app and its dependencies (database, cache) are healthy. Load balancers and orchestrators use them.

```
builder.Services.AddHealthChecks()
    .AddSqlServer(cs, name: "sql")                        // AspNetCore.HealthChecks.SqlServer
    .AddCheck<SftpHealthCheck>("sftp")
    .AddCheck("self", () => HealthCheckResult.Healthy());
app.MapHealthChecks("/health/live", new() { Predicate = c => c.Name == "self" });   // liveness
app.MapHealthChecks("/health/ready");                                               // readiness (dependencies)

public class SftpHealthCheck(ISftpClient sftp) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext ctx,
        CancellationToken ct = default)
        => await sftp.CanConnectAsync(ct) ? HealthCheckResult.Healthy() : HealthCheckResult.Unhealthy("SFTP down");
}
```

⚡ Tricky Point

Keep **liveness** checks trivial (is the process alive?). If liveness checks the database, Kubernetes will restart every pod during a DB outage - making things worse. Put dependency checks in **readiness**.

12.4 Other Useful Features

> ✎ **Definition**
>
> Other ASP.NET Core features you should know about, such as Razor Pages and MVC views, Blazor, output caching, response compression and more.

Feature	What it is
Razor Pages / MVC Views	Server-rendered HTML with Razor syntax
Blazor	C# UI components: Server, WebAssembly, or Auto (interactive render modes in .NET 8+)
.NET Aspire	Opinionated stack for cloud-native apps: orchestration, service discovery, telemetry dashboard
YARP	Reverse proxy / API gateway library
Localization	`IStringLocalizer`, resource files, `UseRequestLocalization`
Data Protection	Encrypting cookies/tokens; key ring management
Output formatters	XML, CSV via custom `OutputFormatter`
`IAsyncEnumerable` streaming	Stream JSON arrays to the client as items are produced
Server-Sent Events	`TypedResults.ServerSentEvents` (.NET 10) for one-way streaming

13. Testing

13.1 Testing Pyramid

> ✎ **Definition**
>
> The **testing pyramid**: many fast **unit tests** at the base, fewer **integration tests** in the middle, and a small number of slow **end-to-end tests** at the top.

Level	What	Tools
Unit tests (many)	One class in isolation; dependencies mocked	xUnit / NUnit / MSTest, Moq / NSubstitute, FluentAssertions / Shouldly
Integration tests	App + real infrastructure (DB, HTTP pipeline)	`WebApplicationFactory`, Testcontainers, Respawn
End-to-end (few)	Full system through the UI/API	Playwright, Postman/Newman, k6

13.2 Unit Testing with xUnit and Moq

> ✎ **Definition**
>
> **Unit testing** checks one class in isolation. **xUnit** is the test framework (Fact, Theory), and **Moq** creates fake (mock) dependencies so behaviour can be controlled and verified.

```
public class ReconServiceTests
{
    private readonly Mock<ITxnRepository> _repo = new();
    private readonly ReconService _sut;
    public ReconServiceTests() => _sut = new ReconService(_repo.Object, NullLogger<ReconService>.Instance);

    [Fact]
    public async Task RunAsync_ReturnsCount_OfUnmatchedTransactions()
    {
        // Arrange
        var date = new DateOnly(2026, 10, 1);
        _repo.Setup(r => r.GetUnmatchedAsync(date)).ReturnsAsync([new Txn("T1"), new Txn("T2")]);
        // Act
        var count = await _sut.RunAsync(date);
        // Assert
        Assert.Equal(2, count);
        _repo.Verify(r => r.GetUnmatchedAsync(date), Times.Once);
    }

    [Theory]
    [InlineData(1000, "VISA", 18)]
    [InlineData(1000, "RUPAY", 0)]
    public void Fee_IsCalculatedPerNetwork(decimal amount, string network, decimal expected)
        => Assert.Equal(expected, new FeeCalculator().Calculate(amount, network));
}
```

> ⚡ **Tricky Point**
>
> Don't mock what you don't own (e.g. `DbContext`, `HttpClient`). Wrap them behind your own interfaces, or test them with integration tests. EF Core's InMemory provider does **not** behave like SQL Server (no constraints, transactions or SQL translation) - prefer SQLite in-memory or Testcontainers with real SQL Server.

> ⚡ **Tricky Point**
>
> **Time and randomness break tests.** Inject `TimeProvider` (.NET 8) instead of calling `DateTime.UtcNow` directly; tests can use `FakeTimeProvider` to control time.

13.3 Integration Testing with WebApplicationFactory

> ✎ **Definition**
>
> **Integration testing** runs the real app in memory with `WebApplicationFactory<Program>` and sends real HTTP requests through the full pipeline, often with a test database.

```
public class TxnApiTests(WebApplicationFactory<Program> factory) : IClassFixture<WebApplicationFactory<Program>>
{
    [Fact]
    public async Task GetById_Returns404_WhenMissing()
    {
        var client = factory.WithWebHostBuilder(b => b.ConfigureTestServices(services =>
        {
            services.RemoveAll<ITxnRepository>();
            services.AddScoped<ITxnRepository, FakeTxnRepository>();
        })).CreateClient();
        var res = await client.GetAsync("/api/transactions/999");
        Assert.Equal(HttpStatusCode.NotFound, res.StatusCode);
    }
}
// For top-level Program.cs add at the end:  public partial class Program { }
```

`WebApplicationFactory` boots the real app in memory with a `TestServer` - routing, middleware, filters, serialization and DI are all exercised without opening a port.

14. Architecture and Design Patterns

14.1 Clean Architecture

> **✎ Definition**
>
> **Clean Architecture** organises code into layers (Domain, Application, Infrastructure, Presentation) where dependencies point **inward**, so business rules don't depend on frameworks or databases.

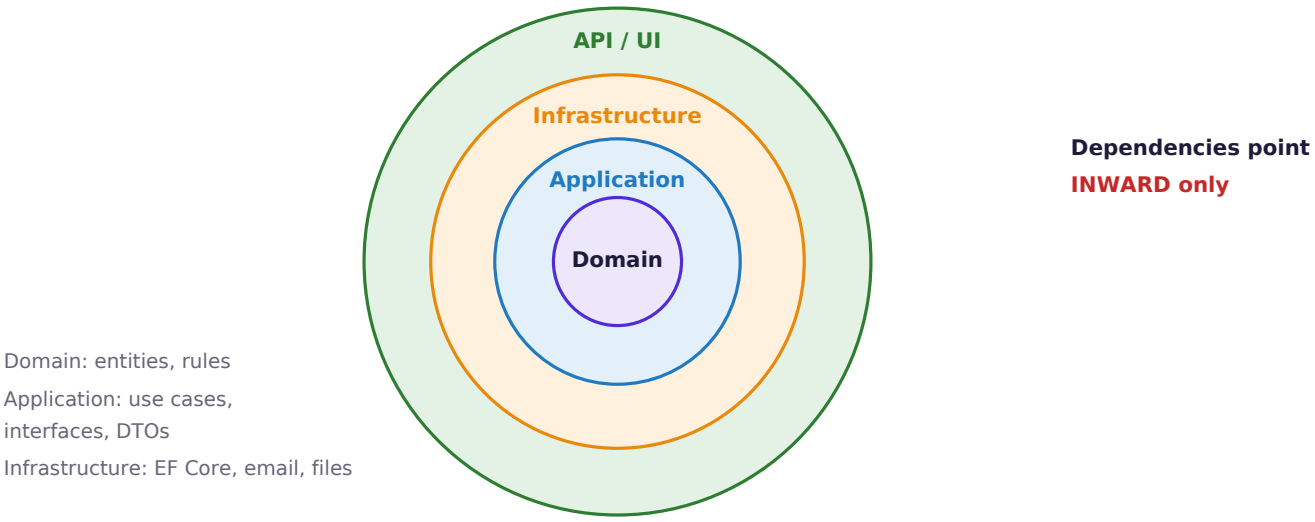


Figure 14.1 - Clean Architecture layers (dependencies point inward)

Project	Contains	Depends on
Recon.Domain	Entities, value objects, domain events, business rules	Nothing
Recon.Application	Use cases (commands/queries), interfaces (`ITxnRepository`, `IEmailSender`), DTOs, validation	Domain
Recon.Infrastructure	EF Core DbContext, repositories, file/SFTP/email/HTTP clients	Application, Domain
Recon.Api	Controllers/minimal APIs, middleware, DI composition root	Application, Infrastructure

> **★ Interview Tip**
>
> The key idea is the **Dependency Rule**: inner layers define interfaces; outer layers implement them. Business logic never references EF Core or ASP.NET Core directly, so it is testable and the database or UI can change without rewriting rules.

> **i Note**
>
> Alternatives: **Vertical Slice Architecture** organises code by feature (each slice has its endpoint, handler, validation and data access together) instead of technical layers - often simpler for CRUD-heavy apps.

14.2 Repository and Unit of Work

✎ Definition

The **Repository** pattern hides data access behind a collection-like interface. **Unit of Work** groups repository changes into one transaction (EF Core's DbContext already implements both).

```
public interface ITxnRepository
{
    Task<Transaction?> GetAsync(long id, CancellationToken ct);
    Task<IReadOnlyList<Transaction>> GetUnmatchedAsync(DateOnly date, CancellationToken ct);
    void Add(Transaction txn);
}
public class TxnRepository(ReconDbContext db) : ITxnRepository
{
    public Task<Transaction?> GetAsync(long id,
        CancellationToken ct) => db.Transactions.FirstOrDefaultAsync(t => t.Id == id, ct);
    public async Task<IReadOnlyList<Transaction>> GetUnmatchedAsync(DateOnly date, CancellationToken ct) =>
        await db.Transactions
            .Where(t => t.Status == TxnStatus.Unmatched && DateOnly.FromDateTime(t.CreatedUtc) == date)
            .ToListAsync(ct);
    public void Add(Transaction txn) => db.Transactions.Add(txn);
}
// implemented by DbContext
public interface IUnitOfWork { Task<int> SaveChangesAsync(CancellationToken ct); }
```

⚡ Tricky Point

`DbContext` already **is** a Unit of Work and `DbSet` already **is** a repository. Wrapping them in generic repositories (`IRepository<T>` with `GetAll()` returning `IEnumerable`) often hides EF's power and causes in-memory filtering. Use specific repositories with intention-revealing methods, or use DbContext directly in application handlers.

14.3 CQRS and MediatR

✎ Definition

CQRS (Command Query Responsibility Segregation) separates writes (commands) from reads (queries). **MediatR** is a library that sends each request to its single handler (the mediator pattern).

CQRS separates **commands** (change state) from **queries** (read state). MediatR dispatches requests to handlers and supports **pipeline behaviors** (cross-cutting concerns like validation, logging, transactions).

```
public record CreateTxnCommand(string Rrn, decimal Amount, int BankId) : IRequest<long>;

public class CreateTxnHandler(ReconDbContext db) : IRequestHandler<CreateTxnCommand, long>
{
    public async Task<long> Handle(CreateTxnCommand cmd, CancellationToken ct)
    {
        var txn = new Transaction { Rrn = cmd.Rrn, Amount = cmd.Amount, BankId = cmd.BankId };
        db.Transactions.Add(txn);
        await db.SaveChangesAsync(ct);
        return txn.Id;
    }
}

public record GetTxnQuery(long Id) : IRequest<TxnDto?>;

public class ValidationBehavior<TReq, TRes>(IEnumerable<IValidator<TReq>> validators)
    : IPipelineBehavior<TReq, TRes> where TReq : notnull
{
    public async Task<TRes> Handle(TReq req, RequestHandlerDelegate<TRes> next, CancellationToken ct)
    {
        var failures = (await Task.WhenAll(validators.Select(v => v.ValidateAsync(req, ct))))
                       .SelectMany(r => r.Errors).ToList();
        if (failures.Count > 0) throw new ValidationException(failures);
        return await next();
    }
}

builder.Services.AddMediatR(c => c.RegisterServicesFromAssemblyContaining<CreateTxnHandler>()
                                  .AddOpenBehavior(typeof(ValidationBehavior<,>)));
// controller
[HttpPost] public async Task<IActionResult> Create(CreateTxnCommand cmd) =>
    CreatedAtAction(nameof(Get), new { id = await mediator.Send(cmd) }, null);
```

i Note

MediatR moved to a commercial licence for newer versions in 2025; teams also use alternatives (Wolverine, Mediator source generator, or plain handler classes injected directly).

14.4 Other Patterns You'll Meet

> **✎ Definition**
>
> Other design patterns common in ASP.NET Core apps, such as Options, Decorator, Strategy, Factory, Specification and Result, and where each is used.

Pattern	Use in ASP.NET Core
Options pattern	Strongly typed configuration
Decorator	Caching/logging wrappers around repositories (Scrutor `Decorate`)
Strategy	Fee calculation per card network, keyed services
Factory	`IHttpClientFactory`, `IDbContextFactory`
Mediator	MediatR request/handler dispatch
Outbox	Save domain events with the DB transaction, publish them reliably later
Saga / process manager	Coordinate long-running distributed workflows
Specification	Reusable query criteria objects
Result pattern	Explicit success/failure without exceptions
Idempotency key	Safe retries of POST requests (payments!)

14.5 Microservices Basics

> **✎ Definition**
>
> **Microservices** is an architecture where an application is split into small, independently deployable services, each owning its data and communicating over HTTP, gRPC or messaging.

- Each service owns its **data**; communicate via HTTP/gRPC (sync) or a message broker (async).
- Use an **API gateway** (YARP, Ocelot, Azure API Management) for routing, auth and rate limiting.
- Make operations **idempotent** and handle **eventual consistency** (outbox pattern, sagas).
- Centralised logging, distributed tracing (OpenTelemetry), health checks, resilience policies.
- Start with a **modular monolith** unless you have clear scaling/team reasons for microservices.

15. Deployment

15.1 Publish Options

✎ Definition

Ways to package an app with `dotnet publish`: **framework-dependent** (needs .NET installed) or **self-contained** (includes the runtime), with options such as single-file, trimming and Native AOT.

```
dotnet publish -c Release -o ./out                                  // framework-dependent
dotnet publish -c Release -r linux-x64 --self-contained             // includes the runtime
dotnet publish -c Release -r win-x64 -p:PublishSingleFile=true      // single executable
dotnet publish -c Release -r linux-x64 -p:PublishAot=true           // Native AOT
// container image without a Dockerfile (.NET 8)
dotnet publish -c Release /t:PublishContainer
```

Mode	Pros	Cons
Framework-dependent	Small output; runtime patched by OS/IT	Runtime must be installed on the server
Self-contained	No runtime install; pinned version	Larger; you must redeploy to patch the runtime
Single file	One file to copy	Larger; some reflection/path caveats
Native AOT	Fast start-up, low memory, no JIT	No runtime code generation; limited reflection, some libraries unsupported (EF Core partial)

15.2 Hosting on IIS (Windows)

✎ Definition

Running ASP.NET Core on Windows **IIS** through the **ASP.NET Core Module**, either in-process (inside the IIS worker process) or out-of-process (IIS as a reverse proxy to Kestrel).

1. Install the **ASP.NET Core Hosting Bundle** (runtime + ASP.NET Core Module) on the server, then restart IIS. 2. Create an App Pool with **.NET CLR version = "No Managed Code"**. 3. Publish to the site folder; `web.config` (generated) configures the ASP.NET Core Module (`hostingModel="inprocess"`). 4. Set environment variables in `web.config` or the app pool, e.g. `ASPNETCORE_ENVIRONMENT=Production`.

```
<aspNetCore processPath="dotnet" arguments=".\Recon.Api.dll" hostingModel="inprocess"
            stdoutLogEnabled="false" stdoutLogFile=".\logs\stdout">
  <environmentVariables>
    <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" />
  </environmentVariables>
</aspNetCore>
```

> ⚡ **Tricky Point**
>
> **HTTP Error 500.30 / 500.31 / 502.5** on IIS usually means the app failed to start: missing Hosting Bundle or runtime version, wrong environment/config, a failing startup (bad connection string, invalid options), or permissions. Temporarily set `stdoutLogEnabled="true"` (create the logs folder) or check the Windows Event Viewer to see the real exception.

> ⚠ **Watch Out**
>
> Deploying over a running in-process app locks DLLs. Drop an `app_offline.htm` file in the site root first (IIS stops the app), copy the files, then delete it. Web Deploy does this automatically.

15.3 Docker

> ✎ **Definition**
>
> **Docker** packages the app and its dependencies into a portable **container image**, built from a Dockerfile (usually multi-stage), so it runs the same way everywhere.

```
# Dockerfile (multi-stage)
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.sln .
COPY src/Recon.Api/*.csproj src/Recon.Api/
RUN dotnet restore
COPY . .
RUN dotnet publish src/Recon.Api -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app .
USER $APP_UID                       # non-root user (.NET 8 images)
ENV ASPNETCORE_HTTP_PORTS=8080      # default port in .NET 8+ images is 8080, not 80
EXPOSE 8080
ENTRYPOINT ["dotnet", "Recon.Api.dll"]
```

```
docker build -t recon-api:1.0 .
docker run -d -p 5000:8080 -e ConnectionStrings__Recon="Server=sql;..." recon-api:1.0
```

> ⚡ **Tricky Point**
>
> Since .NET 8, ASP.NET Core container images listen on port **8080** by default (not 80) and run as a **non-root** user. Old `docker run -p 80:80` setups break after upgrading.

15.4 Cloud and CI/CD

> ✎ **Definition**
>
> Hosting on cloud platforms (Azure App Service, containers, Kubernetes) and **CI/CD**: automated pipelines (GitHub Actions, Azure DevOps) that build, test and deploy on every change.

- **Azure App Service**, **Azure Container Apps**, **AKS** (Kubernetes), **AWS ECS/Elastic Beanstalk/Lambda**.
- CI/CD with GitHub Actions or Azure DevOps: restore, build, test, publish, run DB migration scripts, deploy, smoke-test health endpoints.
- Use deployment slots or blue-green/rolling deployments for zero downtime.

- Configuration via environment variables and Key Vault references; never bake secrets into images.

15.5 Production Readiness Checklist

> ✎ **Definition**
>
> Final checks before going live: configuration and secrets, HTTPS, logging and monitoring, health checks, migrations, resilience, scaling and security hardening.

- `ASPNETCORE_ENVIRONMENT=Production`, HTTPS, HSTS, forwarded headers behind proxies.
- Global exception handler with ProblemDetails; structured logging + correlation IDs; OpenTelemetry tracing.
- Health checks (liveness/readiness); graceful shutdown handling `CancellationToken`.
- Resilience for outbound calls; timeouts everywhere.
- Rate limiting, CORS restricted to known origins, security headers.
- DB migrations via scripts in the pipeline; backups; indexes reviewed.
- Data Protection keys persisted for multi-instance apps.
- Load test critical endpoints; set alerts on error rate and latency.

16. Interview Questions and Tricky Scenarios

16.1 Scenario-Based Puzzles

> **✎ Definition**
>
> Realistic "what goes wrong here?" scenarios, such as wrong middleware order or captive dependencies, each with the cause and the fix. They test practical debugging skill.

Scenario 1 - Middleware order

```
app.UseAuthorization();
app.UseAuthentication();
app.MapControllers();
```

> **⚡ Tricky Point**
>
> Problem: `[Authorize]` endpoints always return **401** even with a valid token, because authorization runs before the user has been authenticated (`HttpContext.User` is still anonymous). Fix: `UseAuthentication()` before `UseAuthorization()`.

Scenario 2 - DbContext in a singleton

```
builder.Services.AddSingleton<ReportCache>();
public class ReportCache(ReconDbContext db) { /* ... */ }
```

> **⚡ Tricky Point**
>
> Problem: captive dependency. In Development the host throws "Cannot consume scoped service from singleton". If validation is disabled, one DbContext is shared across all requests: concurrency exceptions ("A second operation was started on this context instance") and stale data. Fix: inject `IServiceScopeFactory` or `IDbContextFactory<ReconDbContext>`.

Scenario 3 - new HttpClient per call

```
public async Task<string> Call() { using var c = new HttpClient(); return await c.GetStringAsync(url); }
```

> **⚡ Tricky Point**
>
> Problem: under load, sockets stay in TIME_WAIT and the server runs out of ports (`SocketException`). Fix: `IHttpClientFactory` (typed/named clients) or a long-lived `HttpClient` with `SocketsHttpHandler.PooledConnectionLifetime`.

Scenario 4 - Sync over async

```
public IActionResult Get() => Ok(_service.GetDataAsync().Result);
```

> ⚡ **Tricky Point**
>
> Problem: blocks a thread-pool thread for the whole I/O wait; under load causes thread-pool starvation and timeouts. (In ASP.NET Core it doesn't deadlock like classic ASP.NET because there's no SynchronizationContext, but it still kills scalability.) Fix: make the action `async` and `await`.

Scenario 5 - Fire-and-forget with scoped services

```
[HttpPost] public IActionResult Start() { _ = Task.Run(() => _db.Logs.Add(...)); return Accepted(); }
```

> ⚡ **Tricky Point**
>
> Problem: the request scope (and `_db`) is disposed when the response completes, so the background code throws `ObjectDisposedException`; work is also lost on app restart. Fix: queue work to a `BackgroundService` (Channel) or Hangfire, creating its own scope.

Scenario 6 - Exposing entities

```
[HttpGet] public Task<List<Bank>> Get() => _db.Banks.Include(b => b.Transactions).ToListAsync();
```

> ⚡ **Tricky Point**
>
> Problems: serialization cycle (`Bank.Transactions[].Bank...`) -> "A possible object cycle was detected"; returns huge payloads; leaks internal fields. Fix: project to DTOs with `Select`.

Scenario 7 - Missing CancellationToken

> ⚡ **Tricky Point**
>
> A user closes the browser during a 30-second report query. Without passing `HttpContext.RequestAborted` / the action's `CancellationToken` to EF Core, the SQL query keeps running and wastes DB resources. Always accept and forward `CancellationToken`.

Scenario 8 - Configuration not updating

> ⚡ **Tricky Point**
>
> A service injects `IOptions<FeatureFlags>`; you change `appsettings.json` on the server but nothing changes until restart. `IOptions<T>` is computed once. Use `IOptionsMonitor<T>` (or `IOptionsSnapshot<T>` per request) for reloadable settings.

Scenario 9 - CORS still failing

> ⚡ **Tricky Point**
>
> You added `AddCors` and `UseCors`, but the browser still shows a CORS error. Common causes: `UseCors` placed after `MapControllers`/auth, the origin has a trailing slash or a different port, `AllowAnyOrigin` combined with `AllowCredentials`, or the API actually returned a 500 (error responses from the exception handler may lack CORS headers, so the real error is hidden).

Scenario 10 - EF Core loads too much

```
var list = _db.Transactions.ToList().Where(t => t.BankId == id).Take(10);
```

⚡ Tricky Point

Problem: `ToList()` executes `SELECT * FROM Transactions` (millions of rows) before filtering in memory. Fix: keep it `IQueryable` until the end: `Where(...).Take(10).ToListAsync()`.

16.2 Frequently Asked Questions

✎ Definition

The ASP.NET Core theory questions interviewers ask most often, each with a short, precise answer.

Question	Crisp answer
What is .NET Core?	Cross-platform, open-source, high-performance successor of .NET Framework; now just ".NET".
What is Kestrel?	ASP.NET Core's built-in cross-platform web server; often behind IIS/Nginx as reverse proxy.
In-process vs out-of-process IIS hosting?	In-process runs inside w3wp.exe (faster); out-of-process proxies to a separate Kestrel process.
What is middleware?	Components in a pipeline that handle requests/responses, call next or short-circuit; order matters.
Use vs Run vs Map?	Use calls next; Run is terminal; Map branches by path.
DI lifetimes?	Transient: every resolve; Scoped: per request; Singleton: app lifetime.
Captive dependency?	Longer-lived service holding a shorter-lived one (singleton holding scoped).
IOptions vs IOptionsSnapshot vs IOptionsMonitor?	Read once / per request reload (scoped) / live reload with change notifications (singleton).
appsettings precedence?	appsettings.json < appsettings.{Env}.json < user secrets < env vars < command line.
Filters vs middleware?	Filters run within MVC with action context; middleware runs for every request with HttpContext only.
What does [ApiController] do?	Attribute routing required, automatic 400 on invalid model, binding source inference, ProblemDetails.
IActionResult vs ActionResult<T>?	ActionResult<T> lets you return T or a result and documents the type for OpenAPI.
Model binding sources?	Route, query, body, form, header, services.
401 vs 403?	Not authenticated vs authenticated but not permitted.
JWT structure?	header.payload.signature, Base64Url, signed (not encrypted).
Policy-based authorization?	Named rules combining requirements and handlers; supports resource-based checks.
What is CORS?	Browser mechanism allowing cross-origin calls when the server sends allow headers.
EF Core tracking vs no-tracking?	Tracking snapshots entities for updates; AsNoTracking is faster for read-only queries.
N+1 problem?	One query for parents plus one per child (lazy loading); fix with Include/projection.
Eager vs lazy vs explicit loading?	Include up front / proxies on access / Entry(...).Load on demand.
Migrations?	Versioned C# classes describing schema changes; applied via CLI, scripts or bundles.
Optimistic concurrency?	Rowversion/concurrency token; DbUpdateConcurrencyException on conflicting update.

Question	Crisp answer
Why IHttpClientFactory?	Avoids socket exhaustion and stale DNS by pooling/rotating handlers; adds resilience pipelines.
BackgroundService and scoped services?	Hosted services are singletons; create a scope per unit of work.
Caching options?	IMemoryCache, IDistributedCache (Redis), HybridCache, output caching, response caching.
How do you handle errors globally?	UseExceptionHandler + IExceptionHandler + ProblemDetails, with logging and trace IDs.
What is Problem Details?	RFC 9457 standard JSON error format (type, title, status, detail, instance).
Minimal APIs vs controllers?	Lightweight lambda endpoints (fast, AOT-friendly) vs class-based with conventions and filters.
How to secure secrets?	User secrets in dev; environment variables / Key Vault in production.
What is health checking?	Endpoints reporting liveness/readiness for orchestrators and load balancers.
gRPC vs REST?	Binary HTTP/2 contract-first RPC for internal services vs JSON HTTP for public APIs.
What is SignalR?	Real-time server-to-client messaging over WebSockets with fallbacks.
Clean architecture?	Domain/Application at the centre, Infrastructure/UI outside, dependencies pointing inward.
CQRS?	Separate models/handlers for commands (writes) and queries (reads).
Rate limiting?	Built-in middleware (fixed/sliding window, token bucket, concurrency) returning 429.
Graceful shutdown?	Host signals ApplicationStopping and cancels tokens; services finish in-flight work within ShutdownTimeout.

17. Quick Revision Cheat Sheet

17.1 Program.cs Template

✎ Definition

A ready-to-use `Program.cs` for a production-style Web API, with services, middleware and endpoints wired in the correct order.

```
var builder = WebApplication.CreateBuilder(args);
var cfg = builder.Configuration;

builder.Services.AddControllers()
    .AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));
builder.Services.AddOpenApi();
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddDbContext<ReconDbContext>(o => o.UseSqlServer(cfg.GetConnectionString("Recon")));
builder.Services.AddOptions<ReconOptions>().Bind(cfg.GetSection("Recon")).ValidateDataAnnotations()
    .ValidateOnStart();
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(/* ... */);
builder.Services.AddAuthorization();
builder.Services.AddCors(o => o.AddPolicy("Spa", p => p.WithOrigins(cfg["SpaUrl"]!).AllowAnyHeader()
    .AllowAnyMethod()));
builder.Services.AddHybridCache();
builder.Services.AddHttpClient<INpciClient, NpciClient>().AddStandardResilienceHandler();
builder.Services.AddScoped<IReconService, ReconService>();
builder.Services.AddHostedService<ReconWorker>();
builder.Services.AddHealthChecks().AddDbContextCheck<ReconDbContext>();
builder.Services.AddRateLimiter(o => o.AddFixedWindowLimiter("api",
    l => { l.PermitLimit = 100; l.Window = TimeSpan.FromMinutes(1); }));

var app = builder.Build();
app.UseExceptionHandler();
if (app.Environment.IsDevelopment()) app.MapOpenApi(); else app.UseHsts();
app.UseHttpsRedirection();
app.UseCors("Spa");
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.MapControllers().RequireRateLimiting("api");
app.MapHealthChecks("/health");
app.Run();
```

17.2 Top 25 Tricky Points

✎ Definition

The 25 most important ASP.NET Core "gotchas" from this guide, gathered for fast revision before an interview.

- Middleware order matters: exception handler first, authentication before authorization, CORS before endpoints.
- Anything after `app.Run(...)` never executes.
- Scoped services in singletons/hosted services/middleware constructors = captive dependency.
- `DbContext` is not thread-safe; no parallel queries on one instance.
- `IOptions<T>` never reloads; `IOptionsSnapshot<T>` is scoped; `IOptionsMonitor<T>` is live.
- Environment variables use `__` for nesting; unset environment = Production.

- `launchSettings.json` is local only - not deployed.
- Route constraints give 404, not 400.
- `[ApiController]` auto-returns 400 ValidationProblemDetails.
- Only one `[FromBody]` parameter per action.
- 401 = unauthenticated, 403 = forbidden.
- JWT payload is readable by anyone; keep tokens short-lived; ClockSkew default 5 minutes.
- `AllowAnyOrigin` + `AllowCredentials` is invalid; CORS doesn't protect against non-browser clients.

- Never expose EF entities; use DTOs (over-posting, cycles, leaks).
- `ToList()` too early = whole table in memory; keep `IQueryable` until the end.
- Lazy loading causes N+1 queries.
- `FromSqlRaw` with concatenation = SQL injection; use `FromSql($"...")`.
- `ExecuteUpdate/Delete` bypass the change tracker.
- Always configure decimal precision for money columns.
- `new HttpClient()` per request exhausts sockets; typed clients in singletons lose DNS refresh.
- Retry only idempotent operations.
- `.Result`/`.Wait()` in requests cause thread-pool starvation.
- Unhandled exceptions in `BackgroundService.ExecuteAsync` stop the host.
- .NET 8+ container images listen on port 8080 as non-root.
- IIS 500.30 = startup failure: check stdout logs / Event Viewer.

17.3 Study Roadmap

> **✎ Definition**
>
> A suggested order of study, from fundamentals to advanced topics and deployment, with practice tasks for each stage.

Week	Focus	Practice
1	Chapters 1-4: hosting, middleware, DI, configuration, logging	Build an API skeleton with custom middleware, options validation and Serilog
2	Chapters 5-7: controllers, minimal APIs, validation, filters, ProblemDetails	CRUD API for transactions with DTOs, FluentValidation, global error handler
3	Chapter 8: EF Core	Model banks/transactions, migrations, paging, projections, concurrency, Dapper report
4	Chapters 9-10: security, caching, performance	JWT + policies, CORS for a React SPA, rate limiting, HybridCache
5	Chapters 11-13: background jobs, HttpClient, resilience, SignalR, testing	Worker that imports files via SFTP, pushes SignalR updates; unit + integration tests
6	Chapters 14-16: architecture, deployment, interview prep	Refactor to Clean Architecture; Dockerize; deploy to IIS; rehearse scenario answers

> **i Note**
>
> The best way to learn ASP.NET Core is to build one realistic app end-to-end (for example a mini reconciliation API with a React front end) and deliberately apply each chapter to it.